

Stock Portfolio Tracker

Line-by-Line File-Wise Code Explanation

This report covers every human-authored project file: README, SQL schema, PowerShell helper scripts, and all Java source files. Generated .class files and the MySQL connector JAR are intentionally excluded because they are binary/build artifacts, not readable source code.

How to start the project

- Open PowerShell and go to: C:\Users\adris\Desktop\DB Clg proj\stock-market-portfolio-tracker
- Start MySQL with: powershell -ExecutionPolicy Bypass -File .\start-mysql.ps1
- Import/update the database with: Get-Content .\schema.sql | & "C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe" -u root
- Set credentials with: \$env:DB_USER='root' and \$env:DB_PASSWORD=""
- Compile with: powershell -ExecutionPolicy Bypass -File .\compile.ps1
- Run with: powershell -ExecutionPolicy Bypass -File .\run.ps1
- Login using demo@tracker.com / password123 after the schema has been imported.

README.md

Markdown documentation file. The following entries explain each documentation line. Total lines: 100.

Line 1: `# Stock Market Portfolio Tracker`

Explanation: Markdown heading used to organize the README.

Line 2: `[blank line]`

Explanation: Blank line that separates README sections.

Line 3: `A full-stack desktop application built with Java Swing, JDBC, and MySQL for tracking stock market portfolios, executing trades, and viewing simulated market-driven profit/loss changes.`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 4: `[blank line]`

Explanation: Blank line that separates README sections.

Line 5: `> Developed as a 4th-semester Engineering College DBMS + Java project.`

Explanation: Block quote used for a highlighted project note.

Line 6: `[blank line]`

Explanation: Blank line that separates README sections.

Line 7: `---`

Explanation: Bullet point in the documentation.

Line 8: `[blank line]`

Explanation: Blank line that separates README sections.

Line 9: `## Features`

Explanation: Markdown heading used to organize the README.

Line 10: `[blank line]`

Explanation: Blank line that separates README sections.

Line 11: `| Feature | Description |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 12: `|-----|-----|`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 13: `| User Authentication | Secure login and registration with SHA-256 password hashing |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 14: `| Portfolio Dashboard | Create and manage multiple portfolios with auto-refreshing P/L tracking |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 15: `| Transaction Desk | Execute BUY/SELL orders with weighted-average price recalculation |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 16: `| Market Simulator | Dashboard updates latest market prices every 10 seconds |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 17: `[blank line]`

Explanation: Blank line that separates README sections.

Line 18: `## Architecture`

Explanation: Markdown heading used to organize the README.

Line 19: `[blank line]`

Explanation: Blank line that separates README sections.

Line 20: ````text`

Explanation: Starts or ends a fenced code block.

Line 21: `src/com/tracker/`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 22: `■■■ app/ -> Main.java (entry point)`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 23: `■■■ config/ -> DBConnection.java (Singleton JDBC manager)`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 24: `model/ -> POJOs: User, Portfolio, Stock, Holding, Transaction`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 25: `dao/ -> Data Access Objects with PreparedStatements and try-with-resources`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 26: `view/ -> Swing UI: LoginFrame, RegisterFrame, DashboardFrame, TransactionDialog`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 27: `````

Explanation: Starts or ends a fenced code block.

Line 28: [blank line]

Explanation: Blank line that separates README sections.

Line 29: `**Design Patterns Used:** Singleton (DB Connection), DAO Pattern, MVC Separation`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 30: [blank line]

Explanation: Blank line that separates README sections.

Line 31: `## Tech Stack`

Explanation: Markdown heading used to organize the README.

Line 32: [blank line]

Explanation: Blank line that separates README sections.

Line 33: `- **Frontend:** Java Swing`

Explanation: Bullet point in the documentation.

Line 34: `- **Backend:** Core Java, JDBC`

Explanation: Bullet point in the documentation.

Line 35: `- **Database:** MySQL 8.0+`

Explanation: Bullet point in the documentation.

Line 36: `- **Driver:** MySQL Connector/J 8.3.0`

Explanation: Bullet point in the documentation.

Line 37: [blank line]

Explanation: Blank line that separates README sections.

Line 38: `## Database Schema`

Explanation: Markdown heading used to organize the README.

Line 39: [blank line]

Explanation: Blank line that separates README sections.

Line 40: `6 normalized tables with foreign key constraints and `ON DELETE CASCADE`:`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 41: [blank line]

Explanation: Blank line that separates README sections.

Line 42: ````text`

Explanation: Starts or ends a fenced code block.

Line 43: `User -> Portfolio -> Holding <- Stock <- Market_Data`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 44: `Portfolio + Stock -> Transaction`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 45: `````

Explanation: Starts or ends a fenced code block.

Line 46: [blank line]

Explanation: Blank line that separates README sections.

Line 47: `## How to Run`

Explanation: Markdown heading used to organize the README.

Line 48: [blank line]

Explanation: Blank line that separates README sections.

Line 49: `### Prerequisites`

Explanation: Markdown heading used to organize the README.

Line 50: [blank line]

Explanation: Blank line that separates README sections.

Line 51: `- Java 17+ (JDK)`

Explanation: Bullet point in the documentation.

Line 52: `- MySQL 8.0+`

Explanation: Bullet point in the documentation.

Line 53: [blank line]

Explanation: Blank line that separates README sections.

Line 54: `### 1. Start MySQL`

Explanation: Markdown heading used to organize the README.

Line 55: [blank line]

Explanation: Blank line that separates README sections.

Line 56: ````powershell`

Explanation: Starts or ends a fenced code block.

Line 57: `powershell -ExecutionPolicy Bypass -File .\start-mysql.ps1`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 58: `````

Explanation: Starts or ends a fenced code block.

Line 59: [blank line]

Explanation: Blank line that separates README sections.

Line 60: `### 2. Set up the database`

Explanation: Markdown heading used to organize the README.

Line 61: [blank line]

Explanation: Blank line that separates README sections.

Line 62: ````powershell`

Explanation: Starts or ends a fenced code block.

Line 63: `Get-Content .\schema.sql | & "C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe" -u root`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 64: `````

Explanation: Starts or ends a fenced code block.

Line 65: [blank line]

Explanation: Blank line that separates README sections.

Line 66: `### 3. Configure database credentials`

Explanation: Markdown heading used to organize the README.

Line 67: [blank line]

Explanation: Blank line that separates README sections.

Line 68: `The app reads database settings from environment variables:`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 69: [blank line]

Explanation: Blank line that separates README sections.

Line 70: ````powershell`

Explanation: Starts or ends a fenced code block.

Line 71: `$env:DB_URL="jdbc:mysql://localhost:3306/stock_tracker"`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 72: `$env:DB_USER="root"`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 73: `$env:DB_PASSWORD=""`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 74: `````

Explanation: Starts or ends a fenced code block.

Line 75: [blank line]

Explanation: Blank line that separates README sections.

Line 76: `### 4. Compile`

Explanation: Markdown heading used to organize the README.

Line 77: [blank line]

Explanation: Blank line that separates README sections.

Line 78: ````powershell`

Explanation: Starts or ends a fenced code block.

Line 79: `powershell -ExecutionPolicy Bypass -File .\compile.ps1`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 80: `````

Explanation: Starts or ends a fenced code block.

Line 81: [blank line]

Explanation: Blank line that separates README sections.

Line 82: `### 5. Run`

Explanation: Markdown heading used to organize the README.

Line 83: [blank line]

Explanation: Blank line that separates README sections.

Line 84: ````powershell`

Explanation: Starts or ends a fenced code block.

Line 85: `powershell -ExecutionPolicy Bypass -File .\run.ps1`

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 86: `````

Explanation: Starts or ends a fenced code block.

Line 87: [blank line]

Explanation: Blank line that separates README sections.

Line 88: The dashboard updates the latest `Market\_Data` close prices every 10 seconds, then reloads holdings so current price and profit/loss change while the app is open.

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 89: [blank line]

Explanation: Blank line that separates README sections.

Line 90: `## Demo Credentials`

Explanation: Markdown heading used to organize the README.

Line 91: [blank line]

Explanation: Blank line that separates README sections.

Line 92: `| Email | Password |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 93: `|-----|-----|`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 94: `| `demo@tracker.com` | `password123` |`

Explanation: Markdown table row describing a feature, credential, or setup detail.

Line 95: [blank line]

Explanation: Blank line that separates README sections.

Line 96: `## Screens`

Explanation: Markdown heading used to organize the README.

Line 97: [blank line]

Explanation: Blank line that separates README sections.

Line 98: 1. **Login** - Dark themed sign-in with email/password

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 99: 2. **Dashboard** - Portfolio sidebar, holdings table with live P/L, transaction history

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

Line 100: 3. **Trade Dialog** - Stock selector with auto-filled market prices

Explanation: Documentation text explaining project purpose, setup, architecture, or usage.

schema.sql

SQL database setup file. The following entries explain each schema and seed-data line. Total lines: 150.

Line 1: -- =====

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 2: -- Stock Market Portfolio Tracker – Database Setup Script

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 3: -- Run this script in MySQL to create the database and seed data.

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 4: -- =====

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 5: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 6: CREATE DATABASE IF NOT EXISTS stock_tracker;

Explanation: Creates the project database if it does not already exist, so the script can be rerun safely.

Line 7: USE stock_tracker;

Explanation: Selects the stock_tracker database so every following table and insert statement targets it.

Line 8: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 9: -- Remove old anomaly/alert tables if this project was previously initialized

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 10: -- with the earlier anomaly-detection version.

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 11: DROP TABLE IF EXISTS Alert;

Explanation: Part of the database schema or seed data used by the Java application.

Line 12: DROP TABLE IF EXISTS Anomaly;

Explanation: Part of the database schema or seed data used by the Java application.

Line 13: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 14: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 15: -- 1. User table

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 16: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 17: CREATE TABLE IF NOT EXISTS User (

Explanation: Starts a table definition for User.

Line 18: user_id INT PRIMARY KEY AUTO_INCREMENT,

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 19: name VARCHAR(100) NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 20: email VARCHAR(100) NOT NULL UNIQUE,

Explanation: Part of the database schema or seed data used by the Java application.

Line 21: password_hash VARCHAR(255) NOT NULL

Explanation: Part of the database schema or seed data used by the Java application.

Line 22:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 23: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 24: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 25: -- 2. Portfolio table

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 26: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 27: CREATE TABLE IF NOT EXISTS Portfolio (

Explanation: Starts a table definition for Portfolio.

Line 28: portfolio_id INT PRIMARY KEY AUTO_INCREMENT,

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 29: user_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 30: name VARCHAR(100) NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 31: FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE CASCADE

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 32:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 33: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 34: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 35: -- 3. Stock table

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 36: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 37: CREATE TABLE IF NOT EXISTS Stock (

Explanation: Starts a table definition for Stock.

Line 38: stock_id INT PRIMARY KEY AUTO_INCREMENT,

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 39: ticker VARCHAR(10) NOT NULL UNIQUE,

Explanation: Part of the database schema or seed data used by the Java application.

Line 40: company_name VARCHAR(100) NOT NULL

Explanation: Part of the database schema or seed data used by the Java application.

Line 41:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 42: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 43: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 44: -- 4. Holding table

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 45: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 46: CREATE TABLE IF NOT EXISTS Holding (

Explanation: Starts a table definition for Holding.

Line 47: holding_id INT PRIMARY KEY AUTO_INCREMENT,

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 48: portfolio_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 49: stock_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 50: quantity INT NOT NULL DEFAULT 0,

Explanation: Part of the database schema or seed data used by the Java application.

Line 51: avg_buy_price FLOAT NOT NULL DEFAULT 0,

Explanation: Part of the database schema or seed data used by the Java application.

Line 52: FOREIGN KEY (portfolio_id) REFERENCES Portfolio(portfolio_id) ON DELETE CASCADE,

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 53: FOREIGN KEY (stock_id) REFERENCES Stock(stock_id) ON DELETE CASCADE

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 54:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 55: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 56: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 57: -- 5. Transaction table

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 58: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 59: CREATE TABLE IF NOT EXISTS Transaction (

Explanation: Starts a table definition for Transaction.

Line 60: txn_id INT PRIMARY KEY AUTO_INCREMENT,

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 61: portfolio_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 62: stock_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 63: type VARCHAR(10) NOT NULL, -- BUY or SELL

Explanation: Part of the database schema or seed data used by the Java application.

Line 64: quantity INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 65: price FLOAT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 66: timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

Explanation: Part of the database schema or seed data used by the Java application.

Line 67: FOREIGN KEY (portfolio_id) REFERENCES Portfolio(portfolio_id) ON DELETE CASCADE,

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 68: FOREIGN KEY (stock_id) REFERENCES Stock(stock_id) ON DELETE CASCADE

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 69:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 70: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 71: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 72: -- 6. Market_Data table (composite PK)

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 73: -- -----

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 74: CREATE TABLE IF NOT EXISTS Market_Data (

Explanation: Starts a table definition for Market_Data.

Line 75: stock_id INT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 76: date_id DATE NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 77: open_price FLOAT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 78: close_price FLOAT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 79: high FLOAT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 80: low FLOAT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 81: volume BIGINT NOT NULL,

Explanation: Part of the database schema or seed data used by the Java application.

Line 82: PRIMARY KEY (stock_id, date_id),

Explanation: Defines the table's primary key, which uniquely identifies each row.

Line 83: FOREIGN KEY (stock_id) REFERENCES Stock(stock_id) ON DELETE CASCADE

Explanation: Defines a relationship to another table; ON DELETE CASCADE removes dependent rows automatically.

Line 84:) ENGINE=InnoDB;

Explanation: Part of the database schema or seed data used by the Java application.

Line 85: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 86: -- =====

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 87: -- SEED DATA

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 88: -- =====

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 89: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 90: -- Stocks

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 91: INSERT INTO Stock (ticker, company_name) VALUES

Explanation: Begins seed data insertion so the app has demo records to display immediately.

Line 92: ('AAPL', 'Apple Inc.'),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 93: ('TSLA', 'Tesla Inc.'),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 94: ('MSFT', 'Microsoft Corporation'),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 95: ('GOOGL', 'Alphabet Inc.'),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 96: ('AMZN', 'Amazon.com Inc.');

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 97: [blank line]

Explanation: Blank line used to visually separate SQL sections.

```
Line 98: -- Market Data (last 5 trading days for each stock)
```

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

```
Line 99: INSERT INTO Market_Data (stock_id, date_id, open_price, close_price, high, low, volume) VALUES
```

Explanation: Begins seed data insertion so the app has demo records to display immediately.

```
Line 100: -- AAPL (stock_id = 1)
```

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

```
Line 101: (1, '2026-05-12', 188.50, 190.20, 191.00, 187.80, 54000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 102: (1, '2026-05-13', 190.20, 189.10, 191.50, 188.50, 48000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 103: (1, '2026-05-14', 189.10, 192.40, 193.00, 188.90, 62000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 104: (1, '2026-05-15', 192.40, 191.80, 193.50, 190.50, 51000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 105: (1, '2026-05-16', 191.80, 193.50, 194.20, 191.00, 58000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 106: -- TSLA (stock_id = 2)
```

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

```
Line 107: (2, '2026-05-12', 172.30, 175.60, 176.90, 171.00, 98000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 108: (2, '2026-05-13', 175.60, 173.20, 176.00, 172.50, 87000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 109: (2, '2026-05-14', 173.20, 178.90, 179.50, 172.80, 110000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 110: (2, '2026-05-15', 178.90, 176.50, 180.00, 175.00, 95000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 111: (2, '2026-05-16', 176.50, 180.30, 181.50, 175.80, 102000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 112: -- MSFT (stock_id = 3)
```

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

```
Line 113: (3, '2026-05-12', 415.00, 418.30, 420.00, 414.00, 23000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 114: (3, '2026-05-13', 418.30, 416.90, 419.50, 415.50, 20000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 115: (3, '2026-05-14', 416.90, 422.10, 423.00, 416.00, 27000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 116: (3, '2026-05-15', 422.10, 420.50, 424.00, 419.00, 22000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 117: (3, '2026-05-16', 420.50, 425.80, 426.50, 420.00, 25000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 118: -- GOOGL (stock_id = 4)
```

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

```
Line 119: (4, '2026-05-12', 174.20, 176.50, 177.00, 173.50, 30000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 120: (4, '2026-05-13', 176.50, 175.10, 177.20, 174.80, 27000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 121: (4, '2026-05-14', 175.10, 179.30, 180.00, 174.50, 35000000),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 122: (4, '2026-05-15', 179.30, 178.00, 180.50, 177.00, 29000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 123: (4, '2026-05-16', 178.00, 181.20, 182.00, 177.50, 32000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 124: -- AMZN (stock_id = 5)

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 125: (5, '2026-05-12', 185.40, 187.90, 188.50, 184.50, 45000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 126: (5, '2026-05-13', 187.90, 186.30, 188.00, 185.50, 40000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 127: (5, '2026-05-14', 186.30, 190.50, 191.20, 186.00, 52000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 128: (5, '2026-05-15', 190.50, 189.10, 191.50, 188.00, 43000000),

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 129: (5, '2026-05-16', 189.10, 192.80, 193.50, 188.50, 49000000);

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 130: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 131: -- Sample user (password is SHA-256 hash of "password123")

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 132: INSERT INTO User (name, email, password_hash) VALUES

Explanation: Begins seed data insertion so the app has demo records to display immediately.

Line 133: ('Demo User', 'demo@tracker.com', 'ef92b778baf771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f');

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 134: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 135: -- Sample portfolio for demo user

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 136: INSERT INTO Portfolio (user_id, name) VALUES

Explanation: Begins seed data insertion so the app has demo records to display immediately.

Line 137: (1, 'Tech Growth Portfolio');

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 138: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 139: -- Sample holdings

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 140: INSERT INTO Holding (portfolio_id, stock_id, quantity, avg_buy_price) VALUES

Explanation: Begins seed data insertion so the app has demo records to display immediately.

Line 141: (1, 1, 50, 185.00), -- 50 shares of AAPL at avg 185

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 142: (1, 2, 30, 170.00), -- 30 shares of TSLA at avg 170

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 143: (1, 3, 20, 410.00); -- 20 shares of MSFT at avg 410

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

Line 144: [blank line]

Explanation: Blank line used to visually separate SQL sections.

Line 145: -- Sample transactions

Explanation: SQL comment. It documents what the next block of schema or seed data is doing.

Line 146: INSERT INTO Transaction (portfolio_id, stock_id, type, quantity, price, timestamp) VALUES

Explanation: Begins seed data insertion so the app has demo records to display immediately.

```
Line 147: (1, 1, 'BUY', 50, 185.00, '2026-05-10 09:30:00'),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 148: (1, 2, 'BUY', 30, 170.00, '2026-05-10 10:00:00'),
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 149: (1, 3, 'BUY', 20, 410.00, '2026-05-11 11:15:00');
```

Explanation: Seed row or column definition line. In an INSERT block, this is one record being added.

```
Line 150: [blank line]
```

Explanation: Blank line used to visually separate SQL sections.

compile.ps1

PowerShell helper script. The following entries explain each command line. Total lines: 6.

Line 1: `$ErrorActionPreference = "Stop"`

Explanation: Makes PowerShell stop immediately if a command fails, which prevents hidden setup errors.

Line 2: [blank line]

Explanation: Blank line for readability.

Line 3: `$javaFiles = Get-ChildItem -Recurse -Filter *.java -Path "src" | ForEach-Object { $_.FullName }`

Explanation: Finds all Java source files recursively under src and stores their full paths.

Line 4: `javac -d "out" $javaFiles`

Explanation: Finds all Java source files recursively under src and stores their full paths.

Line 5: [blank line]

Explanation: Blank line for readability.

Line 6: `Write-Host "Compiled Java classes into .\out"`

Explanation: Prints a success message after compilation.

run.ps1

PowerShell helper script. The following entries explain each command line. Total lines: 7.

Line 1: `$ErrorActionPreference = "Stop"`

Explanation: Makes PowerShell stop immediately if a command fails, which prevents hidden setup errors.

Line 2: [blank line]

Explanation: Blank line for readability.

Line 3: `if (-not (Test-Path "out")) {`

Explanation: Checks whether compiled output already exists before launching the app.

Line 4: `& ".\compile.ps1"`

Explanation: Runs the compile helper when class files are missing.

Line 5: `}`

Explanation: PowerShell setup/run instruction.

Line 6: [blank line]

Explanation: Blank line for readability.

Line 7: `java -cp "out;lib\mysql-connector-j-8.3.0.jar" com.tracker.app.Main`

Explanation: Starts the Java application and includes both compiled classes and the MySQL JDBC driver on the classpath.

start-mysql.ps1

PowerShell helper script. The following entries explain each command line. Total lines: 23.

Line 1: `$ErrorActionPreference = "Stop"`

Explanation: Makes PowerShell stop immediately if a command fails, which prevents hidden setup errors.

Line 2: [blank line]

Explanation: Blank line for readability.

Line 3: `$mysqlBase = "C:\Program Files\MySQL\MySQL Server 8.4"`

Explanation: PowerShell setup/run instruction.

Line 4: `$dataDir = Join-Path $PSScriptRoot "mysql-data"`

Explanation: PowerShell setup/run instruction.

Line 5: [blank line]

Explanation: Blank line for readability.

Line 6: `if (-not (Test-Path $dataDir)) {`

Explanation: Checks whether compiled output already exists before launching the app.

Line 7: `New-Item -ItemType Directory -Force -Path $dataDir | Out-Null`

Explanation: PowerShell setup/run instruction.

Line 8: `& "$mysqlBase\bin\mysqld.exe" --initialize-insecure --basedir="$mysqlBase" --datadir="$dataDir"`

Explanation: PowerShell setup/run instruction.

Line 9: `}`

Explanation: PowerShell setup/run instruction.

Line 10: [blank line]

Explanation: Blank line for readability.

Line 11: `$running = Get-Process mysqld -ErrorAction SilentlyContinue`

Explanation: PowerShell setup/run instruction.

Line 12: `if (-not $running) {`

Explanation: Checks whether compiled output already exists before launching the app.

Line 13: `$psi = [System.Diagnostics.ProcessStartInfo]::new()`

Explanation: PowerShell setup/run instruction.

Line 14: `$psi.FileName = "$mysqlBase\bin\mysqld.exe"`

Explanation: PowerShell setup/run instruction.

Line 15: `$psi.Arguments = "--basedir=`"$mysqlBase`" --datadir=`"$dataDir`" --port=3306
--log-error=`"$dataDir\server-start.err`"``"`

Explanation: PowerShell setup/run instruction.

Line 16: `$psi.WorkingDirectory = $PSScriptRoot`

Explanation: PowerShell setup/run instruction.

Line 17: `$psi.UseShellExecute = $false`

Explanation: PowerShell setup/run instruction.

Line 18: `$psi.CreateNoWindow = $true`

Explanation: PowerShell setup/run instruction.

Line 19: `[System.Diagnostics.Process]::Start($psi) | Out-Null`

Explanation: PowerShell setup/run instruction.

Line 20: `Start-Sleep -Seconds 5`

Explanation: Pauses the script, used either to wait for MySQL startup or to repeat price simulation every 10 seconds.

Line 21: `}`

Explanation: PowerShell setup/run instruction.

Line 22: [blank line]

Explanation: Blank line for readability.

Line 23: `& "$mysqlBase\bin\mysql.exe" -u root -e "SELECT VERSION();" "`

Explanation: Stores or uses the path to the MySQL command-line client/server executable.

src/com/tracker/app/Main.java

Java source file. The following entries explain each line in order. Total lines: 26.

Line 1: `package com.tracker.app;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.view.LoginFrame;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 5: `import javax.swing.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 6: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 7: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 8: `* Main entry point for the Stock Market Portfolio Tracker application.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 9: `* Sets the system look-and-feel and launches the Login frame on the EDT.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 10: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `public class Main {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 12: `public static void main(String[] args) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 13: `// Set cross-platform look and feel for consistency`

Explanation: Inline comment that explains the intent of the nearby code.

Line 14: `try {`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 15: `UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 16: `} catch (Exception e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 17: `System.err.println("Could not set Look and Feel: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 18: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 19: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 20: `// Launch the UI on the Event Dispatch Thread (EDT)`

Explanation: Inline comment that explains the intent of the nearby code.

Line 21: `SwingUtilities.invokeLater(() -> {`

Explanation: Schedules UI creation on Swing's Event Dispatch Thread, which is the correct thread for Swing components.

Line 22: `LoginFrame loginFrame = new LoginFrame();`

Explanation: Creates or opens the login window.

Line 23: `loginFrame.setVisible(true);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 24: });

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 25: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 26: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/config/DBConnection.java

Java source file. The following entries explain each line in order. Total lines: 73.

Line 1: `package com.tracker.config;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import java.sql.Connection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import java.sql.DriverManager;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: `import java.sql.SQLException;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 6: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 7: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 8: `* Singleton class for managing MySQL database connections using JDBC.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 9: `* Ensures only one connection instance exists throughout the application`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 10: `* lifecycle.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `public class DBConnection {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 13: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 14: `// Database configuration constants – update these for your environment`

Explanation: Inline comment that explains the intent of the nearby code.

Line 15: `private static final String URL = System.getenv().getOrDefault(`

Explanation: Defines the database URL, reading DB_URL from the environment or using the local MySQL default.

Line 16: `"DB_URL", "jdbc:mysql://localhost:3306/stock_tracker");`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 17: `private static final String USER = System.getenv().getOrDefault("DB_USER", "root");`

Explanation: Defines the database username, reading DB_USER from the environment or defaulting to root.

Line 18: `private static final String PASSWORD = System.getenv().getOrDefault("DB_PASSWORD", "");`

Explanation: Defines the database password, reading DB_PASSWORD from the environment or defaulting to blank.

Line 19: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 20: `// Volatile ensures visibility across threads`

Explanation: Inline comment that explains the intent of the nearby code.

Line 21: `private static volatile DBConnection instance;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 22: `private Connection connection;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 23: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 24: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 25: * Private constructor prevents external instantiation.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 26: * Loads the MySQL JDBC driver and establishes a connection.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 27: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 28: private DBConnection() {

Explanation: Declares a private helper method used internally by this class.

Line 29: try {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 30: Class.forName("com.mysql.cj.jdbc.Driver");

Explanation: Loads the MySQL JDBC driver class so DriverManager can create MySQL connections.

Line 31: this.connection = DriverManager.getConnection(URL, USER, PASSWORD);

Explanation: Opens a JDBC connection to MySQL using the configured URL, username, and password.

Line 32: System.out.println("[DBConnection] Connected to MySQL successfully.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 33: } catch (ClassNotFoundException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 34: System.err.println("[DBConnection] MySQL JDBC Driver not found.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 35: e.printStackTrace();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 36: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 37: System.err.println("[DBConnection] Failed to connect to MySQL.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 38: e.printStackTrace();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 39: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 40: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 41: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 42: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 43: * Returns the singleton instance of DBConnection using double-checked locking.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 44: * Thread-safe lazy initialization.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 45: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 46: public static DBConnection getInstance() {

Explanation: Declares a public constructor or method that other classes can call.

Line 47: if (instance == null) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 48: synchronized (DBConnection.class) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 49: if (instance == null) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 50: instance = new DBConnection();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 51: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 52: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 53: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 54: return instance;
```

Explanation: Returns a value from the current method to its caller.

```
Line 55: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 56: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 57: /**
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 58: * Returns the active JDBC Connection object.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 59: * If the connection has been closed or is null, it re-establishes it.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 60: */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 61: public Connection getConnection() {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 62: try {
```

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

```
Line 63: if (connection == null || connection.isClosed()) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 64: System.out.println("[DBConnection] Reconnecting to database...");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 65: connection = DriverManager.getConnection(URL, USER, PASSWORD);
```

Explanation: Opens a JDBC connection to MySQL using the configured URL, username, and password.

```
Line 66: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 67: } catch (SQLException e) {
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 68: System.err.println("[DBConnection] Reconnection failed.");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 69: e.printStackTrace();
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 70: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 71: return connection;
```

Explanation: Returns a value from the current method to its caller.

```
Line 72: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 73: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/model/User.java

Java source file. The following entries explain each line in order. Total lines: 42.

Line 1: `package com.tracker.model;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 4: `* POJO representing a registered user in the system.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 5: `* Maps to the User table in the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 6: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 7: `public class User {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 8: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 9: `private int userId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 10: `private String name;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 11: `private String email;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 12: `private String passwordHash;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 13: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 14: `// Default constructor`

Explanation: Inline comment that explains the intent of the nearby code.

Line 15: `public User() {}`

Explanation: Declares a public constructor or method that other classes can call.

Line 16: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 17: `// Parameterized constructor`

Explanation: Inline comment that explains the intent of the nearby code.

Line 18: `public User(int userId, String name, String email, String passwordHash) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 19: `this.userId = userId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 20: `this.name = name;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 21: `this.email = email;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 22: `this.passwordHash = passwordHash;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 23: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 24: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 25: // Getters and Setters

Explanation: Inline comment that explains the intent of the nearby code.

Line 26: public int getUserId() { return userId; }

Explanation: Declares a public constructor or method that other classes can call.

Line 27: public void setUserId(int userId) { this.userId = userId; }

Explanation: Declares a public constructor or method that other classes can call.

Line 28: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 29: public String getName() { return name; }

Explanation: Declares a public constructor or method that other classes can call.

Line 30: public void setName(String name) { this.name = name; }

Explanation: Declares a public constructor or method that other classes can call.

Line 31: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 32: public String getEmail() { return email; }

Explanation: Declares a public constructor or method that other classes can call.

Line 33: public void setEmail(String email) { this.email = email; }

Explanation: Declares a public constructor or method that other classes can call.

Line 34: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 35: public String getPasswordHash() { return passwordHash; }

Explanation: Declares a public constructor or method that other classes can call.

Line 36: public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }

Explanation: Declares a public constructor or method that other classes can call.

Line 37: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 38: @Override

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 39: public String toString() {

Explanation: Declares a public constructor or method that other classes can call.

Line 40: return "User{userId=" + userId + ", name='" + name + "', email='" + email + "'}";

Explanation: Returns a value from the current method to its caller.

Line 41: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 42: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/model/Portfolio.java

Java source file. The following entries explain each line in order. Total lines: 35.

Line 1: `package com.tracker.model;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 4: `* POJO representing a user's investment portfolio.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 5: `* Maps to the Portfolio table in the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 6: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 7: `public class Portfolio {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 8: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 9: `private int portfolioId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 10: `private int userId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 11: `private String name;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 12: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 13: `public Portfolio() {}`

Explanation: Declares a public constructor or method that other classes can call.

Line 14: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 15: `public Portfolio(int portfolioId, int userId, String name) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 16: `this.portfolioId = portfolioId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 17: `this.userId = userId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 18: `this.name = name;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 19: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 20: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 21: `// Getters and Setters`

Explanation: Inline comment that explains the intent of the nearby code.

Line 22: `public int getPortfolioId() { return portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 23: `public void setPortfolioId(int portfolioId) { this.portfolioId = portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 24: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 25: `public int getUserId() { return userId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 26: `public void setUserId(int userId) { this.userId = userId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 27: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 28: `public String getName() { return name; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 29: `public void setName(String name) { this.name = name; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 30: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 31: `@Override`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 32: `public String toString() {`

Explanation: Declares a public constructor or method that other classes can call.

Line 33: `return name; // So JComboBox / JList displays the portfolio name directly`

Explanation: Returns a value from the current method to its caller.

Line 34: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 35: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/model/Stock.java

Java source file. The following entries explain each line in order. Total lines: 35.

Line 1: `package com.tracker.model;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 4: `* POJO representing a stock / listed security.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 5: `* Maps to the Stock table in the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 6: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 7: `public class Stock {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 8: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 9: `private int stockId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 10: `private String ticker;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 11: `private String companyName;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 12: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 13: `public Stock() {}`

Explanation: Declares a public constructor or method that other classes can call.

Line 14: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 15: `public Stock(int stockId, String ticker, String companyName) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 16: `this.stockId = stockId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 17: `this.ticker = ticker;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 18: `this.companyName = companyName;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 19: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 20: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 21: `// Getters and Setters`

Explanation: Inline comment that explains the intent of the nearby code.

Line 22: `public int getStockId() { return stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 23: `public void setStockId(int stockId) { this.stockId = stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 24: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 25: `public String getTicker() { return ticker; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 26: `public void setTicker(String ticker) { this.ticker = ticker; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 27: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 28: `public String getCompanyName() { return companyName; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 29: `public void setCompanyName(String companyName) { this.companyName = companyName; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 30: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 31: `@Override`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 32: `public String toString() {`

Explanation: Declares a public constructor or method that other classes can call.

Line 33: `return ticker + " - " + companyName;`

Explanation: Returns a value from the current method to its caller.

Line 34: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 35: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/model/Holding.java

Java source file. The following entries explain each line in order. Total lines: 67.

Line 1: `package com.tracker.model;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 4: `* POJO representing a stock holding within a portfolio.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 5: `* Maps to the Holding table in the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 6: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 7: `public class Holding {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 8: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 9: `private int holdingId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 10: `private int portfolioId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 11: `private int stockId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 12: `private int quantity;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 13: `private float avgBuyPrice;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 14: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 15: `// Additional display-only fields (not stored in the Holding table)`

Explanation: Inline comment that explains the intent of the nearby code.

Line 16: `private String ticker;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 17: `private String companyName;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 18: `private float currentPrice; // Latest close_price from Market_Data`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 19: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 20: `public Holding() {}`

Explanation: Declares a public constructor or method that other classes can call.

Line 21: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 22: `public Holding(int holdingId, int portfolioId, int stockId, int quantity, float avgBuyPrice) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 23: `this.holdingId = holdingId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 24: `this.portfolioId = portfolioId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 25: `this.stockId = stockId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 26: `this.quantity = quantity;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 27: `this.avgBuyPrice = avgBuyPrice;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 28: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 29: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 30: `// Getters and Setters`

Explanation: Inline comment that explains the intent of the nearby code.

Line 31: `public int getHoldingId() { return holdingId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 32: `public void setHoldingId(int holdingId) { this.holdingId = holdingId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 33: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 34: `public int getPortfolioId() { return portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 35: `public void setPortfolioId(int portfolioId) { this.portfolioId = portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 36: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 37: `public int getStockId() { return stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 38: `public void setStockId(int stockId) { this.stockId = stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 39: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 40: `public int getQuantity() { return quantity; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 41: `public void setQuantity(int quantity) { this.quantity = quantity; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 42: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 43: `public float getAvgBuyPrice() { return avgBuyPrice; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 44: `public void setAvgBuyPrice(float avgBuyPrice) { this.avgBuyPrice = avgBuyPrice; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 45: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 46: `public String getTicker() { return ticker; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 47: `public void setTicker(String ticker) { this.ticker = ticker; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 48: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 49: public String getCompanyName() { return companyName; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 50: public void setCompanyName(String companyName) { this.companyName = companyName; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 51: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 52: public float getCurrentPrice() { return currentPrice; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 53: public void setCurrentPrice(float currentPrice) { this.currentPrice = currentPrice; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 54: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 55: /**
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 56: * Calculates the total profit/loss for this holding.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 57: * P/L = (currentPrice - avgBuyPrice) * quantity
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 58: */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 59: public float getProfitLoss() {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 60: return (currentPrice - avgBuyPrice) * quantity;
```

Explanation: Returns a value from the current method to its caller.

```
Line 61: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 62: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 63: @Override
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 64: public String toString() {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 65: return "Holding{ticker='" + ticker + "', qty=" + quantity + ", avgPrice=" + avgBuyPrice + "}";
```

Explanation: Returns a value from the current method to its caller.

```
Line 66: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 67: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/model/Transaction.java

Java source file. The following entries explain each line in order. Total lines: 64.

Line 1: `package com.tracker.model;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import java.sql.Timestamp;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 5: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 6: `* POJO representing a BUY/SELL transaction record.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 7: `* Maps to the Transaction table in the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 8: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 9: `public class Transaction {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 10: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 11: `private int txnId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 12: `private int portfolioId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 13: `private int stockId;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 14: `private String type; // "BUY" or "SELL"`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 15: `private int quantity;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 16: `private float price;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 17: `private Timestamp timestamp;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 18: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 19: `// Display-only helper field`

Explanation: Inline comment that explains the intent of the nearby code.

Line 20: `private String ticker;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 21: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 22: `public Transaction() {}`

Explanation: Declares a public constructor or method that other classes can call.

Line 23: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 24: `public Transaction(int txnId, int portfolioId, int stockId,`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 25: `String type, int quantity, float price, Timestamp timestamp) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 26: `this.txnId = txnId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 27: `this.portfolioId = portfolioId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 28: `this.stockId = stockId;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 29: `this.type = type;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 30: `this.quantity = quantity;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 31: `this.price = price;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 32: `this.timestamp = timestamp;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 33: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 34: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 35: `// Getters and Setters`

Explanation: Inline comment that explains the intent of the nearby code.

Line 36: `public int getTxnId() { return txnId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 37: `public void setTxnId(int txnId) { this.txnId = txnId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 38: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 39: `public int getPortfolioId() { return portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 40: `public void setPortfolioId(int portfolioId) { this.portfolioId = portfolioId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 41: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 42: `public int getStockId() { return stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 43: `public void setStockId(int stockId) { this.stockId = stockId; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 44: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 45: `public String getType() { return type; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 46: `public void setType(String type) { this.type = type; }`

Explanation: Declares a public constructor or method that other classes can call.

Line 47: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 48: `public int getQuantity() { return quantity; }`

Explanation: Declares a public constructor or method that other classes can call.

```
Line 49: public void setQuantity(int quantity) { this.quantity = quantity; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 50: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 51: public float getPrice() { return price; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 52: public void setPrice(float price) { this.price = price; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 53: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 54: public Timestamp getTimestamp() { return timestamp; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 55: public void setTimestamp(Timestamp timestamp) { this.timestamp = timestamp; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 56: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 57: public String getTicker() { return ticker; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 58: public void setTicker(String ticker) { this.ticker = ticker; }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 59: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 60: @Override
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 61: public String toString() {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 62: return "Txn{" + type + " " + quantity + " x " + ticker + " @ " + price + "}";
```

Explanation: Returns a value from the current method to its caller.

```
Line 63: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 64: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/dao/UserDAO.java

Java source file. The following entries explain each line in order. Total lines: 96.

Line 1: `package com.tracker.dao;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.config.DBConnection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.User;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 6: `import java.security.MessageDigest;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 7: `import java.security.NoSuchAlgorithmException;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import java.sql.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 10: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `* Data Access Object for User-related database operations.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `* Handles registration, authentication, and user lookup.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 13: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 14: `public class UserDAO {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 15: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 16: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 17: `* Registers a new user. The password is hashed using SHA-256 before storage.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 18: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 19: `* @param name Full name of the user`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `* @param email Email address (must be unique)`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `* @param password Raw password (will be hashed)`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 22: `* @return true if registration succeeds, false otherwise`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 23: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 24: `public boolean registerUser(String name, String email, String password) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 25: `String sql = "INSERT INTO User (name, email, password_hash) VALUES (?, ?, ?)";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 26: `String hash = hashPassword(password);`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 27: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 28: `try (Connection conn = DBConnection.getInstance().getConnection());`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 29: `PreparedStatement ps = conn.prepareStatement(sql) {`

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 30: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 31: `ps.setString(1, name);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 32: `ps.setString(2, email);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 33: `ps.setString(3, hash);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 34: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 35: `int rows = ps.executeUpdate();`

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 36: `return rows > 0;`

Explanation: Returns a value from the current method to its caller.

Line 37: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 38: `} catch (SQLException e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 39: `// Duplicate email will throw an integrity constraint violation`

Explanation: Inline comment that explains the intent of the nearby code.

Line 40: `System.err.println("[UserDAO] Registration failed: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 41: `return false;`

Explanation: Returns a value from the current method to its caller.

Line 42: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 43: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 44: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 45: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 46: `* Authenticates a user by email and password.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 47: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 48: `* @param email User's email`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 49: * @param password Raw password to verify
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 50: * @return User object if credentials match, null otherwise
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 51: */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 52: public User authenticate(String email, String password) {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 53: String sql = "SELECT * FROM User WHERE email = ? AND password_hash = ?";
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 54: String hash = hashPassword(password);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 55: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 56: try (Connection conn = DBConnection.getInstance().getConnection();
```

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

```
Line 57: PreparedStatement ps = conn.prepareStatement(sql)) {
```

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

```
Line 58: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 59: ps.setString(1, email);
```

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

```
Line 60: ps.setString(2, hash);
```

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

```
Line 61: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 62: try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

```
Line 63: if (rs.next()) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 64: User user = new User();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 65: user.setUserId(rs.getInt("user_id"));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 66: user.setName(rs.getString("name"));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 67: user.setEmail(rs.getString("email"));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 68: user.setPasswordHash(rs.getString("password_hash"));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 69: return user;
```

Explanation: Returns a value from the current method to its caller.

```
Line 70: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 71: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 72: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

Line 73: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 74: System.err.println("[UserDAO] Authentication error: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 75: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 76: return null;

Explanation: Returns a value from the current method to its caller.

Line 77: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 78: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 79: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 80: * Hashes a password using the SHA-256 algorithm.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 81: * Produces a 64-character hex string identical to the seed data format.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 82: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 83: private String hashPassword(String password) {

Explanation: Declares a private helper method used internally by this class.

Line 84: try {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 85: MessageDigest md = MessageDigest.getInstance("SHA-256");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 86: byte[] digest = md.digest(password.getBytes());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 87: StringBuilder sb = new StringBuilder();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 88: for (byte b : digest) {

Explanation: Loops over a collection or array to process multiple items.

Line 89: sb.append(String.format("%02x", b));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 90: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 91: return sb.toString();

Explanation: Returns a value from the current method to its caller.

Line 92: } catch (NoSuchAlgorithmException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 93: throw new RuntimeException("SHA-256 algorithm not available", e);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 94: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 95: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 96: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/dao/PortfolioDAO.java

Java source file. The following entries explain each line in order. Total lines: 70.

Line 1: `package com.tracker.dao;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.config.DBConnection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.Portfolio;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 6: `import java.sql.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 7: `import java.util.ArrayList;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import java.util.List;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 10: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `* Data Access Object for Portfolio-related database operations.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `* Supports listing, creation, and lookup of portfolios.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 13: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 14: `public class PortfolioDAO {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 15: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 16: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 17: `* Retrieves all portfolios belonging to a specific user.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 18: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 19: `* @param userId The ID of the logged-in user`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `* @return List of Portfolio objects`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 22: `public List<Portfolio> getPortfoliosByUser(int userId) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 23: `List<Portfolio> list = new ArrayList<>();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 24: String sql = "SELECT * FROM Portfolio WHERE user_id = ?";

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 25: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 26: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 27: PreparedStatement ps = conn.prepareStatement(sql) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 28: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 29: ps.setInt(1, userId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 30: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 31: try (ResultSet rs = ps.executeQuery()) {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 32: while (rs.next()) {

Explanation: Loops while a condition remains true, commonly while reading ResultSet rows.

Line 33: Portfolio p = new Portfolio();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 34: p.setPortfolioId(rs.getInt("portfolio_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 35: p.setUserId(rs.getInt("user_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 36: p.setName(rs.getString("name"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 37: list.add(p);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 38: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 39: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 40: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 41: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 42: System.err.println("[PortfolioDAO] Error fetching portfolios: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 43: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 44: return list;

Explanation: Returns a value from the current method to its caller.

Line 45: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 46: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 47: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 48: * Creates a new portfolio for the given user.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 49: *	
Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 50: * @param userId The owner's user ID	
Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 51: * @param name Portfolio name	
Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 52: * @return true if creation succeeds	
Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 53: */	
Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.	
Line 54: public boolean createPortfolio(int userId, String name) {	
Explanation: Declares a public constructor or method that other classes can call.	
Line 55: String sql = "INSERT INTO Portfolio (user_id, name) VALUES (?, ?)";	
Explanation: Assigns or initializes a value used by the surrounding method or class.	
Line 56: [blank line]	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 57: try (Connection conn = DBConnection.getInstance().getConnection();	
Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.	
Line 58: PreparedStatement ps = conn.prepareStatement(sql) {	
Explanation: Uses a PreparedStatement to run parameterized SQL safely.	
Line 59: [blank line]	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 60: ps.setInt(1, userId);	
Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.	
Line 61: ps.setString(2, name);	
Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.	
Line 62: [blank line]	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 63: return ps.executeUpdate() > 0;	
Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.	
Line 64: [blank line]	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 65: } catch (SQLException e) {	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 66: System.err.println("[PortfolioDAO] Error creating portfolio: " + e.getMessage());	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 67: return false;	
Explanation: Returns a value from the current method to its caller.	
Line 68: }	
Explanation: Closes or opens a Java block, method, class, lambda, or initializer.	
Line 69: }	
Explanation: Closes or opens a Java block, method, class, lambda, or initializer.	
Line 70: }	
Explanation: Closes or opens a Java block, method, class, lambda, or initializer.	

src/com/tracker/dao/StockDAO.java

Java source file. The following entries explain each line in order. Total lines: 101.

Line 1: `package com.tracker.dao;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.config.DBConnection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.Stock;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 6: `import java.sql.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 7: `import java.util.ArrayList;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import java.util.List;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 10: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `* Data Access Object for Stock-related database operations.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `* Provides methods to list all stocks and retrieve by ID or ticker.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 13: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 14: `public class StockDAO {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 15: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 16: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 17: `* Fetches all available stocks from the Stock table.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 18: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 19: `* @return List of all Stock objects`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `public List<Stock> getAllStocks() {`

Explanation: Declares a public constructor or method that other classes can call.

Line 22: `List<Stock> list = new ArrayList<>();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 23: `String sql = "SELECT * FROM Stock ORDER BY ticker";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 24: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 25: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 26: PreparedStatement ps = conn.prepareStatement(sql);

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 27: ResultSet rs = ps.executeQuery() {

Explanation: Works with rows returned by a SQL SELECT query.

Line 28: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 29: while (rs.next()) {

Explanation: Loops while a condition remains true, commonly while reading ResultSet rows.

Line 30: Stock s = new Stock();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 31: s.setStockId(rs.getInt("stock_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 32: s.setTicker(rs.getString("ticker"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 33: s.setCompanyName(rs.getString("company_name"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 34: list.add(s);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 35: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 36: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 37: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 38: System.err.println("[StockDAO] Error fetching stocks: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 39: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 40: return list;

Explanation: Returns a value from the current method to its caller.

Line 41: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 42: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 43: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 44: * Fetches the latest close price for a given stock from Market_Data.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 45: * Uses ORDER BY date_id DESC LIMIT 1 to get the most recent record.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 46: *

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 47: * @param stockId The stock's primary key

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 48: * @return The latest close price, or 0.0f if not found

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 49: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 50: public float getLatestClosePrice(int stockId) {

Explanation: Declares a public constructor or method that other classes can call.

Line 51: String sql = "SELECT close_price FROM Market_Data WHERE stock_id = ? ORDER BY date_id DESC LIMIT 1";

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 52: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 53: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 54: PreparedStatement ps = conn.prepareStatement(sql) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 55: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 56: ps.setInt(1, stockId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 57: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 58: try (ResultSet rs = ps.executeQuery()) {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 59: if (rs.next()) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 60: return rs.getFloat("close_price");

Explanation: Returns a value from the current method to its caller.

Line 61: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 62: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 63: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 64: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 65: System.err.println("[StockDAO] Error fetching price: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 66: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 67: return 0.0f;

Explanation: Returns a value from the current method to its caller.

Line 68: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 69: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 70: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 71: * Simulates market movement by updating the latest Market_Data row for every

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 72: * stock. This keeps the dashboard's current price and P/L values moving

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 73: * without needing a real market-data API.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 74: *

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 75: * @return true when at least one stock price was updated

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 76: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 77: public boolean simulateLatestPriceMovement() {

Explanation: Declares a public constructor or method that other classes can call.

Line 78: String sql = "UPDATE Market_Data md "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 79: + "JOIN ("

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 80: + " SELECT stock_id, MAX(date_id) AS latest_date "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 81: + " FROM Market_Data "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 82: + " GROUP BY stock_id "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 83: + ") latest ON md.stock_id = latest.stock_id "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 84: + " AND md.date_id = latest.latest_date "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 85: + "SET md.open_price = md.close_price, "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 86: + " md.close_price = ROUND(GREATEST(1, md.close_price * (1 + ((RAND() - 0.5) / 20))), 2), "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 87: + " md.high = GREATEST(md.high, md.close_price), "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 88: + " md.low = LEAST(md.low, md.close_price), "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 89: + " md.volume = GREATEST(1000, ROUND(md.volume * (1 + ((RAND() - 0.5) / 8))))";

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 90: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 91: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 92: PreparedStatement ps = conn.prepareStatement(sql) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 93: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 94: return ps.executeUpdate() > 0;

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 95: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 96: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 97: System.err.println("[StockDAO] Error simulating price movement: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 98: `return false;`

Explanation: Returns a value from the current method to its caller.

Line 99: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 100: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 101: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/dao/HoldingDAO.java

Java source file. The following entries explain each line in order. Total lines: 150.

Line 1: `package com.tracker.dao;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.config.DBConnection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.Holding;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 6: `import java.sql.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 7: `import java.util.ArrayList;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import java.util.List;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 10: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 11: `* Data Access Object for Holding-related database operations.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `* Manages portfolio holdings – fetching, inserting, updating, and deleting.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 13: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 14: `public class HoldingDAO {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 15: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 16: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 17: `* Fetches all holdings for a given portfolio, enriched with stock ticker,`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 18: `* company name, and the latest close price from Market_Data.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 19: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `* @param portfolioId The portfolio to query`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `* @return List of enriched Holding objects`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 22: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 23: `public List<Holding> getHoldingsByPortfolio(int portfolioId) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 24: `List<Holding> list = new ArrayList<>();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 25: `String sql = "SELECT h.*, s.ticker, s.company_name, "`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 26: `+ "(SELECT md.close_price FROM Market_Data md "`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 27: `+ " WHERE md.stock_id = h.stock_id ORDER BY md.date_id DESC LIMIT 1) AS current_price "`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 28: `+ "FROM Holding h "`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 29: `+ "JOIN Stock s ON h.stock_id = s.stock_id "`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 30: `+ "WHERE h.portfolio_id = ?";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 31: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 32: `try (Connection conn = DBConnection.getInstance().getConnection());`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 33: `PreparedStatement ps = conn.prepareStatement(sql)) {`

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 34: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 35: `ps.setInt(1, portfolioId);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 36: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 37: `try (ResultSet rs = ps.executeQuery()) {`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 38: `while (rs.next()) {`

Explanation: Loops while a condition remains true, commonly while reading ResultSet rows.

Line 39: `Holding h = new Holding();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 40: `h.setHoldingId(rs.getInt("holding_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 41: `h.setPortfolioId(rs.getInt("portfolio_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 42: `h.setStockId(rs.getInt("stock_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 43: `h.setQuantity(rs.getInt("quantity"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 44: `h.setAvgBuyPrice(rs.getFloat("avg_buy_price"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 45: `h.setTicker(rs.getString("ticker"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 46: `h.setCompanyName(rs.getString("company_name"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 47: `h.setCurrentPrice(rs.getFloat("current_price"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 48: `list.add(h);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 49: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 50: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 51: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 52: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 53: System.err.println("[HoldingDAO] Error fetching holdings: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 54: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 55: return list;

Explanation: Returns a value from the current method to its caller.

Line 56: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 57: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 58: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 59: * Finds an existing holding for a given portfolio and stock combination.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 60: *

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 61: * @return The Holding if found, null otherwise

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 62: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 63: public Holding findHolding(int portfolioId, int stockId) {

Explanation: Declares a public constructor or method that other classes can call.

Line 64: String sql = "SELECT * FROM Holding WHERE portfolio_id = ? AND stock_id = ?";

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 65: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 66: try (Connection conn = DBConnection.getInstance().getConnection();

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 67: PreparedStatement ps = conn.prepareStatement(sql)) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 68: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 69: ps.setInt(1, portfolioId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 70: ps.setInt(2, stockId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 71: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 72: try (ResultSet rs = ps.executeQuery()) {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 73: `if (rs.next()) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 74: `Holding h = new Holding();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 75: `h.setHoldingId(rs.getInt("holding_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 76: `h.setPortfolioId(rs.getInt("portfolio_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 77: `h.setStockId(rs.getInt("stock_id"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 78: `h.setQuantity(rs.getInt("quantity"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 79: `h.setAvgBuyPrice(rs.getFloat("avg_buy_price"));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 80: `return h;`

Explanation: Returns a value from the current method to its caller.

Line 81: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 82: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 83: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 84: `} catch (SQLException e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 85: `System.err.println("[HoldingDAO] Error finding holding: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 86: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 87: `return null;`

Explanation: Returns a value from the current method to its caller.

Line 88: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 89: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 90: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 91: `* Inserts a new holding record into the database.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 92: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 93: `public boolean insertHolding(int portfolioId, int stockId, int quantity, float avgPrice) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 94: `String sql = "INSERT INTO Holding (portfolio_id, stock_id, quantity, avg_buy_price) VALUES (?, ?, ?, ?)";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 95: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 96: `try (Connection conn = DBConnection.getInstance().getConnection();`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 97: `PreparedStatement ps = conn.prepareStatement(sql) {`

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 98: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 99: `ps.setInt(1, portfolioId);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 100: `ps.setInt(2, stockId);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 101: `ps.setInt(3, quantity);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 102: `ps.setFloat(4, avgPrice);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 103: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 104: `return ps.executeUpdate() > 0;`

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 105: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 106: `} catch (SQLException e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 107: `System.err.println("[HoldingDAO] Error inserting holding: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 108: `return false;`

Explanation: Returns a value from the current method to its caller.

Line 109: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 110: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 111: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 112: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 113: `* Updates an existing holding's quantity and average buy price.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 114: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 115: `public boolean updateHolding(int holdingId, int newQuantity, float newAvgPrice) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 116: `String sql = "UPDATE Holding SET quantity = ?, avg_buy_price = ? WHERE holding_id = ?";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 117: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 118: `try (Connection conn = DBConnection.getInstance().getConnection();`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 119: `PreparedStatement ps = conn.prepareStatement(sql)) {`

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 120: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 121: `ps.setInt(1, newQuantity);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 122: `ps.setFloat(2, newAvgPrice);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 123: `ps.setInt(3, holdingId);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 124: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 125: `return ps.executeUpdate() > 0;`

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 126: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 127: `} catch (SQLException e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 128: `System.err.println("[HoldingDAO] Error updating holding: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 129: `return false;`

Explanation: Returns a value from the current method to its caller.

Line 130: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 131: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 132: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 133: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 134: `* Deletes a holding record (used when quantity reaches zero after selling).`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 135: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 136: `public boolean deleteHolding(int holdingId) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 137: `String sql = "DELETE FROM Holding WHERE holding_id = ?";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 138: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 139: `try (Connection conn = DBConnection.getInstance().getConnection();`

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 140: `PreparedStatement ps = conn.prepareStatement(sql)) {`

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 141: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 142: `ps.setInt(1, holdingId);`

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 143: `return ps.executeUpdate() > 0;`

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 144: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 145: `} catch (SQLException e) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 146: `System.err.println("[HoldingDAO] Error deleting holding: " + e.getMessage());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 147: `return false;`

Explanation: Returns a value from the current method to its caller.

Line 148: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 149: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 150: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/dao/TransactionDAO.java

Java source file. The following entries explain each line in order. Total lines: 149.

Line 1: `package com.tracker.dao;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.config.DBConnection;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.Holding;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: `import com.tracker.model.Transaction;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 6: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 7: `import java.sql.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import java.util.ArrayList;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: `import java.util.List;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 10: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 11: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 12: `* Data Access Object for Transaction-related database operations.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 13: `* Implements BUY and SELL transaction logic with holding management.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 14: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 15: `public class TransactionDAO {`

Explanation: Declares a Java class. This is the main type defined by this file.

Line 16: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 17: `private final HoldingDAO holdingDAO = new HoldingDAO();`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 18: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 19: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `* Executes a BUY transaction:`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `* 1. Logs the transaction in the Transaction table.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 22: `* 2. If the user already holds this stock in the portfolio,`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 23: `* updates the quantity and recalculates the weighted average buy price.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 24: * 3. If not, creates a new Holding entry.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 25: *

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 26: * @param portfolioId Portfolio to buy into

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 27: * @param stockId Stock to purchase

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 28: * @param quantity Number of shares

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 29: * @param price Price per share

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 30: * @return true if the buy succeeds

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 31: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 32: public boolean executeBuy(int portfolioId, int stockId, int quantity, float price) {

Explanation: Declares a public constructor or method that other classes can call.

Line 33: // Step 1: Log the transaction

Explanation: Inline comment that explains the intent of the nearby code.

Line 34: if (!logTransaction(portfolioId, stockId, "BUY", quantity, price)) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 35: return false;

Explanation: Returns a value from the current method to its caller.

Line 36: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 37: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 38: // Step 2: Update or create the holding

Explanation: Inline comment that explains the intent of the nearby code.

Line 39: Holding existing = holdingDAO.findHolding(portfolioId, stockId);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 40: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 41: if (existing != null) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 42: // Recalculate weighted average price:

Explanation: Inline comment that explains the intent of the nearby code.

Line 43: // newAvg = (oldQty * oldAvg + newQty * newPrice) / (oldQty + newQty)

Explanation: Inline comment that explains the intent of the nearby code.

Line 44: int totalQty = existing.getQuantity() + quantity;

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 45: float newAvg = ((existing.getQuantity() * existing.getAvgBuyPrice())

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 46: + (quantity * price)) / totalQty;

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 47: return holdingDAO.updateHolding(existing.getHoldingId(), totalQty, newAvg);

Explanation: Returns a value from the current method to its caller.

Line 48: } else {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 49: return holdingDAO.insertHolding(portfolioId, stockId, quantity, price);
```

Explanation: Returns a value from the current method to its caller.

```
Line 50: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 51: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 52: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 53: /**
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 54: * Executes a SELL transaction:
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 55: * 1. Validates that the user holds enough shares.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 56: * 2. Logs the transaction in the Transaction table.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 57: * 3. Decreases the holding quantity (or deletes the holding if qty reaches 0).
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 58: *
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 59: * @return true if the sell succeeds, false if insufficient shares or error
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 60: */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 61: public boolean executeSell(int portfolioId, int stockId, int quantity, float price) {
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 62: // Step 1: Validate sufficient holding
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 63: Holding existing = holdingDAO.findHolding(portfolioId, stockId);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 64: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 65: if (existing == null || existing.getQuantity() < quantity) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 66: System.err.println("[TransactionDAO] Insufficient shares to sell.");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 67: return false;
```

Explanation: Returns a value from the current method to its caller.

```
Line 68: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 69: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 70: // Step 2: Log the transaction
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 71: if (!logTransaction(portfolioId, stockId, "SELL", quantity, price)) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 72: return false;
```

Explanation: Returns a value from the current method to its caller.

Line 73: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 74: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 75: // Step 3: Update the holding

Explanation: Inline comment that explains the intent of the nearby code.

Line 76: int remainingQty = existing.getQuantity() - quantity;

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 77: if (remainingQty == 0) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 78: return holdingDAO.deleteHolding(existing.getHoldingId());

Explanation: Returns a value from the current method to its caller.

Line 79: } else {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 80: // Average buy price remains unchanged after selling

Explanation: Inline comment that explains the intent of the nearby code.

Line 81: return holdingDAO.updateHolding(existing.getHoldingId(), remainingQty, existing.getAvgBuyPrice());

Explanation: Returns a value from the current method to its caller.

Line 82: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 83: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 84: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 85: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 86: * Retrieves recent transactions for a given portfolio, ordered newest-first.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 87: * Joins with Stock table to include ticker symbols.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 88: *

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 89: * @param portfolioId Portfolio to query

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 90: * @return List of Transaction objects (most recent 50)

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 91: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 92: public List<Transaction> getRecentTransactions(int portfolioId) {

Explanation: Declares a public constructor or method that other classes can call.

Line 93: List<Transaction> list = new ArrayList<>();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 94: String sql = "SELECT t.*, s.ticker FROM Transaction t "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 95: + "JOIN Stock s ON t.stock_id = s.stock_id "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 96: + "WHERE t.portfolio_id = ? "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 97: + "ORDER BY t.timestamp DESC LIMIT 50";

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 98: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 99: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 100: PreparedStatement ps = conn.prepareStatement(sql) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 101: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 102: ps.setInt(1, portfolioId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 103: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 104: try (ResultSet rs = ps.executeQuery()) {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 105: while (rs.next()) {

Explanation: Loops while a condition remains true, commonly while reading ResultSet rows.

Line 106: Transaction txn = new Transaction();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 107: txn.setTxnId(rs.getInt("txn_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 108: txn.setPortfolioId(rs.getInt("portfolio_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 109: txn.setStockId(rs.getInt("stock_id"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 110: txn.setType(rs.getString("type"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 111: txn.setQuantity(rs.getInt("quantity"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 112: txn.setPrice(rs.getFloat("price"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 113: txn.setTimestamp(rs.getTimestamp("timestamp"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 114: txn.setTicker(rs.getString("ticker"));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 115: list.add(txn);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 116: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 117: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 118: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 119: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 120: System.err.println("[TransactionDAO] Error fetching transactions: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 121: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 122: return list;

Explanation: Returns a value from the current method to its caller.

Line 123: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 124: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 125: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 126: * Internal helper to insert a record into the Transaction table.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 127: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 128: private boolean logTransaction(int portfolioId, int stockId,

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 129: String type, int quantity, float price) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 130: String sql = "INSERT INTO Transaction (portfolio_id, stock_id, type, quantity, price, timestamp) "

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 131: + "VALUES (?, ?, ?, ?, ?, NOW());"

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 132: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 133: try (Connection conn = DBConnection.getInstance().getConnection());

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 134: PreparedStatement ps = conn.prepareStatement(sql)) {

Explanation: Uses a PreparedStatement to run parameterized SQL safely.

Line 135: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 136: ps.setInt(1, portfolioId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 137: ps.setInt(2, stockId);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 138: ps.setString(3, type);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 139: ps.setInt(4, quantity);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 140: ps.setFloat(5, price);

Explanation: Binds a Java value into a PreparedStatement placeholder before executing SQL.

Line 141: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 142: return ps.executeUpdate() > 0;

Explanation: Runs an INSERT, UPDATE, or DELETE and returns how many rows changed.

Line 143: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 144: } catch (SQLException e) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 145: System.err.println("[TransactionDAO] Error logging transaction: " + e.getMessage());

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 146: return false;

Explanation: Returns a value from the current method to its caller.

Line 147: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 148: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 149: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Explanation: Declares a public constructor or method that other classes can call.

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Sets the font family, size, or weight for a Swing component.

Explanation: Sets a Swing component's text color or background color.

Explanation: Sets a Swing component's text color or background color.

Explanation: Sets border/padding styling around a Swing component.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Sets the font family, size, or weight for a Swing component.

Explanation: Sets a Swing component's text color or background color.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Returns a value from the current method to its caller.

Line 196: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 197: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 198: private JTextField createTextField() {

Explanation: Declares a private helper method used internally by this class.

Line 199: JTextField field = new JTextField();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 200: styleField(field);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 201: return field;

Explanation: Returns a value from the current method to its caller.

Line 202: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 203: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 204: private void styleField(JTextField field) {

Explanation: Declares a private helper method used internally by this class.

Line 205: field.setFont(new Font("Segoe UI", Font.PLAIN, 14));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 206: field.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 207: field.setBackground(FIELD_BG);

Explanation: Sets a Swing component's text color or background color.

Line 208: field.setCaretColor(TEXT_PRIMARY);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 209: field.setBorder(BorderFactory.createCompoundBorder(

Explanation: Sets border/padding styling around a Swing component.

Line 210: BorderFactory.createLineBorder(FIELD_BORDER),

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 211: new EmptyBorder(8, 12, 8, 12)

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 212:));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 213: field.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 214: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 215: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 24: `private static final Color FIELD_BORDER = new Color(70, 75, 90);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 25: `private static final Color SUCCESS_GREEN = new Color(80, 200, 120);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 26: `private static final Color ERROR_RED = new Color(255, 90, 90);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 27: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 28: `private JTextField nameField;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 29: `private JTextField emailField;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 30: `private JPasswordField passwordField;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 31: `private JPasswordField confirmPassword;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 32: `private JButton registerButton;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 33: `private JLabel statusLabel;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 34: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 35: `private final LoginFrame parentLogin;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 36: `private final UserDao userDao = new UserDao();`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 37: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 38: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 39: `* @param parentLogin Reference to the login frame so we can re-show it on cancel/success.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 40: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 41: `public RegisterFrame(LoginFrame parentLogin) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 42: `this.parentLogin = parentLogin;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 43: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 44: `setTitle("Stock Tracker – Register");`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 45: `setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 46: `setSize(460, 620);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 47: `setLocationRelativeTo(null);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 48: `setResizable(false);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 49: `getContentPane().setBackground(BG_DARK);`

Explanation: Sets a Swing component's text color or background color.

Line 50: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 51: `// When the user closes this window, show the login frame again`

Explanation: Inline comment that explains the intent of the nearby code.

Line 52: `addWindowListener(new WindowAdapter() {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 53: `@Override`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 54: `public void windowClosing(WindowEvent e) {`

Explanation: Declares a public constructor or method that other classes can call.

Line 55: `parentLogin.setVisible(true);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 56: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 57: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 58: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 59: `initComponents();`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 60: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 61: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 62: `private void initComponents() {`

Explanation: Declares a private helper method used internally by this class.

Line 63: `JPanel card = new JPanel();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 64: `card.setLayout(new BorderLayout(card, BorderLayout.Y_AXIS));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 65: `card.setBackground(PANEL_BG);`

Explanation: Sets a Swing component's text color or background color.

Line 66: `card.setBorder(BorderFactory.createCompoundBorder(`

Explanation: Sets border/padding styling around a Swing component.

Line 67: `BorderFactory.createLineBorder(FIELD_BORDER, 1),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 68: `new EmptyBorder(36, 40, 36, 40)`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 69: `));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 70: `card.setMaximumSize(new Dimension(380, 540));`

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 71: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 72: `// Title`

Explanation: Inline comment that explains the intent of the nearby code.

Line 73: JLabel title = new JLabel("Create Account");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 74: title.setFont(new Font("Segoe UI", Font.BOLD, 22));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 75: title.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 76: title.setAlignmentX(Component.CENTER_ALIGNMENT);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 77: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 78: JLabel subtitle = new JLabel("Join the portfolio tracker");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 79: subtitle.setFont(new Font("Segoe UI", Font.PLAIN, 13));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 80: subtitle.setForeground(TEXT_SECONDARY);

Explanation: Sets a Swing component's text color or background color.

Line 81: subtitle.setAlignmentX(Component.CENTER_ALIGNMENT);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 82: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 83: // Fields

Explanation: Inline comment that explains the intent of the nearby code.

Line 84: nameField = createTextField();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 85: emailField = createTextField();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 86: passwordField = new JPasswordField(); styleField(passwordField);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 87: confirmPassword = new JPasswordField(); styleField(confirmField);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 88: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 89: // Register button

Explanation: Inline comment that explains the intent of the nearby code.

Line 90: registerButton = new JButton("Create Account");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 91: registerButton.setFont(new Font("Segoe UI", Font.BOLD, 14));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 92: registerButton.setForeground(Color.WHITE);

Explanation: Sets a Swing component's text color or background color.

Line 93: registerButton.setBackground(ACCENT);

Explanation: Sets a Swing component's text color or background color.

Line 94: registerButton.setFocusPainted(false);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 95: registerButton.setBorderPainted(false);

Explanation: Sets border/padding styling around a Swing component.

Line 96: registerButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 97: registerButton.setMaximumSize(new Dimension(Integer.MAX_VALUE, 42));

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 98: `registerButton.setAlignmentX(Component.CENTER_ALIGNMENT);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 99: `registerButton.addMouseListener(new MouseAdapter() {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 100: `public void mouseEntered(MouseEvent e) { registerButton.setBackground(ACCENT_HOVER); }`

Explanation: Declares a public constructor or method that other classes can call.

Line 101: `public void mouseExited(MouseEvent e) { registerButton.setBackground(ACCENT); }`

Explanation: Declares a public constructor or method that other classes can call.

Line 102: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 103: `registerButton.addActionListener(e -> handleRegister());`

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Line 104: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 105: `// Back to login link`

Explanation: Inline comment that explains the intent of the nearby code.

Line 106: `JButton backButton = new JButton("Already have an account? Sign In");`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 107: `backButton.setFont(new Font("Segoe UI", Font.PLAIN, 12));`

Explanation: Sets the font family, size, or weight for a Swing component.

Line 108: `backButton.setForeground(ACCENT);`

Explanation: Sets a Swing component's text color or background color.

Line 109: `backButton.setBackground(PANEL_BG);`

Explanation: Sets a Swing component's text color or background color.

Line 110: `backButton.setBorderPainted(false);`

Explanation: Sets border/padding styling around a Swing component.

Line 111: `backButton.setFocusPainted(false);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 112: `backButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 113: `backButton.setAlignmentX(Component.CENTER_ALIGNMENT);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 114: `backButton.addActionListener(e -> {`

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Line 115: `parentLogin.setVisible(true);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 116: `dispose();`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 117: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 118: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 119: `// Status`

Explanation: Inline comment that explains the intent of the nearby code.

Line 120: `statusLabel = new JLabel(" ");`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 121: `statusLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));`

Explanation: Sets the font family, size, or weight for a Swing component.

Line 122: `statusLabel.setForeground(ERROR_RED);`

Explanation: Sets a Swing component's text color or background color.

Line 123: `statusLabel.setAlignmentX(Component.CENTER_ALIGNMENT);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 124: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 125: `// Assembly`

Explanation: Inline comment that explains the intent of the nearby code.

Line 126: `card.add(title);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 127: `card.add(Box.createVerticalStrut(4));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 128: `card.add(subtitle);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 129: `card.add(Box.createVerticalStrut(24));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 130: `card.add(createLabel("Full Name")); card.add(Box.createVerticalStrut(6)); card.add(nameField);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 131: `card.add(Box.createVerticalStrut(14));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 132: `card.add(createLabel("Email Address")); card.add(Box.createVerticalStrut(6)); card.add(emailField);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 133: `card.add(Box.createVerticalStrut(14));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 134: `card.add(createLabel("Password")); card.add(Box.createVerticalStrut(6)); card.add(passwordField);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 135: `card.add(Box.createVerticalStrut(14));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 136: `card.add(createLabel("Confirm Password")); card.add(Box.createVerticalStrut(6)); card.add(confirmField);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 137: `card.add(Box.createVerticalStrut(24));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 138: `card.add(registerButton);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 139: `card.add(Box.createVerticalStrut(10));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 140: `card.add(statusLabel);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 141: `card.add(Box.createVerticalStrut(8));`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 142: `card.add(backButton);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 143: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 144: `JPanel wrapper = new JPanel(new GridBagLayout());`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 145: `wrapper.setBackground(BG_DARK);`

Explanation: Sets a Swing component's text color or background color.

Line 146: `wrapper.add(card);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 147: setContentPane(wrapper);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 148: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 149: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 150: /**
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 151: * Validates all fields and registers the user via UserDAO.
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 152: */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 153: private void handleRegister() {
```

Explanation: Declares a private helper method used internally by this class.

```
Line 154: String name = nameField.getText().trim();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 155: String email = emailField.getText().trim();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 156: String pass = new String(passwordField.getPassword());
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 157: String confirm = new String(confirmField.getPassword());
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 158: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 159: // Validation
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 160: if (name.isEmpty() || email.isEmpty() || pass.isEmpty()) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 161: showError("All fields are required.");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 162: return;
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 163: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 164: if (!email.contains("@") || !email.contains(".")) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 165: showError("Please enter a valid email address.");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 166: return;
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 167: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 168: if (pass.length() < 6) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 169: showError("Password must be at least 6 characters.");
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 170: return;
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 171: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 172: if (!pass.equals(confirm)) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 173: showError("Passwords do not match.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 174: return;

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 175: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 176: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 177: boolean success = userDao.registerUser(name, email, pass);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 178: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 179: if (success) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 180: statusLabel.setForeground(SUCCESS_GREEN);

Explanation: Sets a Swing component's text color or background color.

Line 181: statusLabel.setText("Account created! Redirecting to login...");

Explanation: Updates visible text in a label, button, or field.

Line 182: Timer timer = new Timer(1200, evt -> {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 183: parentLogin.setVisible(true);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 184: dispose();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 185: });

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 186: timer.setRepeats(false);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 187: timer.start();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 188: } else {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 189: showError("Registration failed. Email may already be in use.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 190: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 191: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 192: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 193: private void showError(String msg) {

Explanation: Declares a private helper method used internally by this class.

Line 194: statusLabel.setForeground(ERROR_RED);

Explanation: Sets a Swing component's text color or background color.

Line 195: statusLabel.setText(msg);

```
Line 196: }
```

```
Line 197: [blank line]
```

[illegible]

```
Line 199: private JLabel createLabel(String text) {
```

```
Line 200: JLabel l = new JLabel(text);
```

```
Line 201: l.setFont(new Font("Segoe UI", Font.PLAIN, 12));
```

```
Line 202: l.setForeground(TEXT_SECONDARY);
```

```
Line 203: l.setAlignmentX(Component.LEFT_ALIGNMENT);
```

```
Line 204: return l;
```

```
Line 205: }
```

Line 206: [blank line]

```
Line 207: private JTextField createTextField() {
```

```
Line 208: JTextField field = new JTextField();
```

```
Line 209: styleField(field);
```

```
Line 210: return field;
```

```
Line 211: }
```

```
Line 212: [blank line]
```

```
Line 213: private void styleField(JTextField field) {
```

```
Line 214: field.setFont(new Font("Segoe UI", Font.PLAIN, 14));
```

```
Line 215: field.setForeground(TEXT_PRIMARY);
```

```
Line 216: field.setBackground(FIELD_BG);
```

```
Line 217: field.setCaretColor(TEXT_PRIMARY);
```

```
Line 218: field.setBorder(BorderFactory.createCompoundBorder(
```

```
Line 219: BorderFactory.createLineBorder(FIELD_BORDER),
```

Page 79

Line 220: `new EmptyBorder(8, 12, 8, 12)`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 221: `));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 222: `field.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));`

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 223: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 224: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

src/com/tracker/view/DashboardFrame.java

Java source file. The following entries explain each line in order. Total lines: 471.

Line 1: `package com.tracker.view;`

Explanation: Declares the Java package, which maps this file into the project's folder/module structure.

Line 2: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 3: `import com.tracker.dao.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 4: `import com.tracker.model.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 5: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 6: `import javax.swing.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 7: `import javax.swing.border.EmptyBorder;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 8: `import javax.swing.table.DefaultTableCellRenderer;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 9: `import javax.swing.table.DefaultTableModel;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 10: `import javax.swing.table.JTableHeader;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 11: `import java.awt.*;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 12: `import java.util.List;`

Explanation: Imports a Java or project class so it can be used by its short name in this file.

Line 13: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 14: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 15: `* Main dashboard displayed after successful login.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 16: `* Contains four major sections:`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 17: `* 1. Portfolio selector (left sidebar)`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 18: `* 2. Holdings table (center)`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 19: `* 3. Recent transactions table (center-bottom)`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 20: `*`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 21: `* Uses BorderLayout and nested panels with proper layout managers – no null layouts.`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 22: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 23: `public class DashboardFrame extends JFrame {`

Explanation: Declares a Java class. This is the main type defined by this file.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Declares a private helper method used internally by this class.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Declares a private helper method used internally by this class.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Sets a Swing component's text color or background color.

Explanation: Sets border/padding styling around a Swing component.

Stock Portfolio Tracker - Line-by-Line Explanation

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 98: // Left: App title
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 99: JLabel appTitle = new JLabel("\u25C8 Stock Portfolio Tracker");
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 100: appTitle.setFont(new Font("Segoe UI", Font.BOLD, 18));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 101: appTitle.setForeground(TEXT_PRIMARY);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 102: header.add(appTitle, BorderLayout.WEST);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 103: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 104: // Right: User info + Logout
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 105: JPanel rightPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 0));
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 106: rightPanel.setOpaque(false);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 107: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 108: JLabel userLabel = new JLabel("Signed in as " + currentUser.getName());
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 109: userLabel.setFont(new Font("Segoe UI", Font.PLAIN, 13));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 110: userLabel.setForeground(TEXT_SECONDARY);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 111: rightPanel.add(userLabel);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 112: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 113: JButton logoutBtn = createStyledButton("Logout", new Color(200, 60, 60), new Color(230, 80, 80));
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 114: logoutBtn.addActionListener(e -> {
```

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

```
Line 115: new LoginFrame().setVisible(true);
```

Explanation: Creates or opens the login window.

```
Line 116: dispose();
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 117: });
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 118: rightPanel.add(logoutBtn);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 119: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 120: header.add(rightPanel, BorderLayout.EAST);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 121: return header;
```

Explanation: Returns a value from the current method to its caller.

Line 122: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 123: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 124: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 125: // SIDEBAR – Portfolio list

Explanation: Inline comment that explains the intent of the nearby code.

Line 126: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 127: private JPanel createSidebar() {

Explanation: Declares a private helper method used internally by this class.

Line 128: JPanel sidebar = new JPanel(new BorderLayout());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 129: sidebar.setPreferredSize(new Dimension(230, 0));

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 130: sidebar.setBackground(SIDEBAR_BG);

Explanation: Sets a Swing component's text color or background color.

Line 131: sidebar.setBorder(BorderFactory.createMatteBorder(0, 0, 0, 1, BORDER_COLOR));

Explanation: Sets border/padding styling around a Swing component.

Line 132: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 133: // Title

Explanation: Inline comment that explains the intent of the nearby code.

Line 134: JLabel sideTitle = new JLabel(" My Portfolios");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 135: sideTitle.setFont(new Font("Segoe UI", Font.BOLD, 14));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 136: sideTitle.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 137: sideTitle.setBorder(new EmptyBorder(14, 10, 10, 10));

Explanation: Sets border/padding styling around a Swing component.

Line 138: sidebar.add(sideTitle, BorderLayout.NORTH);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 139: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 140: // Portfolio list

Explanation: Inline comment that explains the intent of the nearby code.

Line 141: portfolioListModel = new DefaultListModel<>();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 142: portfolioList = new JList<>(portfolioListModel);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 143: portfolioList.setBackground(SIDEBAR_BG);

Explanation: Sets a Swing component's text color or background color.

Line 144: portfolioList.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 145: portfolioList.setSelectionBackground(ACCENT);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 146: portfolioList.setSelectionForeground(Color.WHITE);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 147: portfolioList.setFont(new Font("Segoe UI", Font.PLAIN, 13));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 148: portfolioList.setFixedCellHeight(38);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 149: portfolioList.setBorder(new EmptyBorder(4, 10, 4, 10));
```

Explanation: Sets border/padding styling around a Swing component.

```
Line 150: portfolioList.addListSelectionListener(e -> {
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 151: if (!e.getValueIsAdjusting()) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 152: selectedPortfolio = portfolioList.getSelectedValue();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 153: if (selectedPortfolio != null) {
```

Explanation: Starts a conditional branch that only runs when the stated condition is true.

```
Line 154: refreshDashboard();
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 155: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 156: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 157: });
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 158: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 159: JScrollPane listScroll = new JScrollPane(portfolioList);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 160: listScroll.setBorder(BorderFactory.createEmptyBorder());
```

Explanation: Sets border/padding styling around a Swing component.

```
Line 161: listScroll.getViewport().setBackground(SIDEBAR_BG);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 162: sidebar.add(listScroll, BorderLayout.CENTER);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

```
Line 163: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 164: // "New Portfolio" button at bottom
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 165: JButton newPortfolioBtn = createStyledButton("+ New Portfolio", ACCENT, ACCENT_HOVER);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 166: newPortfolioBtn.addActionListener(e -> createNewPortfolio());
```

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

```
Line 167: JPanel btnWrap = new JPanel(new BorderLayout());
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 168: btnWrap.setBackground(SIDEBAR_BG);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 169: btnWrap.setBorder(new EmptyBorder(8, 12, 12, 12));
```

Explanation: Sets border/padding styling around a Swing component.

```
Line 170: btnWrap.add(newPortfolioBtn, BorderLayout.CENTER);
```

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Blank line used to separate logical parts of the Java file.

Line 196: `JButton tradeBtn = createStyledButton("New Trade", ACCENT, ACCENT_HOVER);`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 197: `tradeBtn.addActionListener(e -> openTradeDialog());`

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Line 198: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 199: `JButton refreshBtn = createStyledButton("Refresh", new Color(60, 65, 80), new Color(75, 80, 95));`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 200: `refreshBtn.addActionListener(e -> refreshDashboard());`

Explanation: Connects a button, combo box, or radio button action to code that should run when the user interacts with it.

Line 201: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 202: `summaryStrip.add(totalValueLabel);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 203: `summaryStrip.add(totalPLLabel);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 204: `summaryStrip.add(tradeBtn);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 205: `summaryStrip.add(refreshBtn);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 206: `center.add(summaryStrip, BorderLayout.NORTH);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 207: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 208: `// ■■■ Split: Holdings (top) / Transactions (bottom) ■■■■`

Explanation: Inline comment that explains the intent of the nearby code.

Line 209: `JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT);`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 210: `splitPane.setResizeWeight(0.6);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 211: `splitPane.setDividerSize(4);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 212: `splitPane.setBackground(BG_DARK);`

Explanation: Sets a Swing component's text color or background color.

Line 213: `splitPane.setBorder(null);`

Explanation: Sets border/padding styling around a Swing component.

Line 214: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 215: `splitPane.setTopComponent(createHoldingsPanel());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 216: `splitPane.setBottomComponent(createTransactionsPanel());`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 217: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 218: `center.add(splitPane, BorderLayout.CENTER);`

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 219: `return center;`

Explanation: Returns a value from the current method to its caller.

Line 220: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 221: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 222: /** Holdings table panel */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 223: private JPanel createHoldingsPanel() {

Explanation: Declares a private helper method used internally by this class.

Line 224: JPanel panel = new JPanel(new BorderLayout());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 225: panel.setBackground(PANEL_BG);

Explanation: Sets a Swing component's text color or background color.

Line 226: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 227: JLabel label = new JLabel(" Holdings");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 228: label.setFont(new Font("Segoe UI", Font.BOLD, 13));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 229: label.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 230: label.setBorder(new EmptyBorder(10, 8, 6, 0));

Explanation: Sets border/padding styling around a Swing component.

Line 231: panel.add(label, BorderLayout.NORTH);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 232: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 233: String[] cols = {"Ticker", "Company", "Qty", "Avg Buy Price", "Current Price", "P/L", "P/L %"};

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 234: holdingsTableModel = new DefaultTableModel(cols, 0) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 235: @Override

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 236: public boolean isCellEditable(int row, int col) { return false; }

Explanation: Declares a public constructor or method that other classes can call.

Line 237: };

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 238: holdingsTable = createStyledTable(holdingsTableModel);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 239: panel.add(new JScrollPane(holdingsTable) {{

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 240: setBorder(null);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 241: getViewport().setBackground(TABLE_ROW);

Explanation: Sets a Swing component's text color or background color.

Line 242: }}, BorderLayout.CENTER);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 243: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 244: return panel;

Explanation: Returns a value from the current method to its caller.

Line 245: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 246: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 247: /** Transactions table panel */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 248: private JPanel createTransactionsPanel() {

Explanation: Declares a private helper method used internally by this class.

Line 249: JPanel panel = new JPanel(new BorderLayout());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 250: panel.setBackground(PANEL_BG);

Explanation: Sets a Swing component's text color or background color.

Line 251: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 252: JLabel label = new JLabel(" Recent Transactions");

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 253: label.setFont(new Font("Segoe UI", Font.BOLD, 13));

Explanation: Sets the font family, size, or weight for a Swing component.

Line 254: label.setForeground(TEXT_PRIMARY);

Explanation: Sets a Swing component's text color or background color.

Line 255: label.setBorder(new EmptyBorder(10, 8, 6, 0));

Explanation: Sets border/padding styling around a Swing component.

Line 256: panel.add(label, BorderLayout.NORTH);

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 257: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 258: String[] cols = {"ID", "Ticker", "Type", "Qty", "Price", "Timestamp"};

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 259: txnTableModel = new DefaultTableModel(cols, 0) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 260: @Override

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 261: public boolean isCellEditable(int row, int col) { return false; }

Explanation: Declares a public constructor or method that other classes can call.

Line 262: };

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 263: txnTable = createStyledTable(txnTableModel);

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 264: panel.add(new JScrollPane(txnTable) { {

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Line 265: setBorder(null);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 266: getViewport().setBackground(TABLE_ROW);

Explanation: Sets a Swing component's text color or background color.

Line 267: }}, BorderLayout.CENTER);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 268: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 269: return panel;

Explanation: Returns a value from the current method to its caller.

Line 270: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 271: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 272: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 273: // DATA LOADING

Explanation: Inline comment that explains the intent of the nearby code.

Line 274: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 275: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 276: /** Load portfolios into the sidebar list. */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 277: private void loadPortfolios() {

Explanation: Declares a private helper method used internally by this class.

Line 278: portfolioListModel.clear();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 279: List<Portfolio> portfolios = portfolioDAO.getPortfoliosByUser(currentUser.getUserId());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 280: for (Portfolio p : portfolios) {

Explanation: Loops over a collection or array to process multiple items.

Line 281: portfolioListModel.addElement(p);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 282: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 283: if (!portfolios.isEmpty()) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 284: portfolioList.setSelectedIndex(0);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 285: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 286: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 287: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 288: /** Refresh holdings and transactions for the selected portfolio. */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 289: private void refreshDashboard() {

Explanation: Declares a private helper method used internally by this class.

Line 290: if (selectedPortfolio == null) return;

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 291: loadHoldings();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 292: loadTransactions();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 293: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 294: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 295: `/** Populate the holdings table and compute totals. */`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 296: `private void loadHoldings() {`

Explanation: Declares a private helper method used internally by this class.

Line 297: `holdingsTableModel.setRowCount(0);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 298: `List<Holding> holdings = holdingDAO.getHoldingsByPortfolio(selectedPortfolio.getPortfolioId());`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 299: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 300: `float totalValue = 0;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 301: `float totalPL = 0;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 302: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 303: `for (Holding h : holdings) {`

Explanation: Loops over a collection or array to process multiple items.

Line 304: `float pl = h.getProfitLoss();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 305: `float plPercent = h.getAvgBuyPrice() != 0`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 306: `? ((h.getCurrentPrice() - h.getAvgBuyPrice()) / h.getAvgBuyPrice()) * 100`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 307: `: 0;`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 308: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 309: `holdingsTableModel.addRow(new Object[]{`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 310: `h.getTicker(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 311: `h.getCompanyName(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 312: `h.getQuantity(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 313: `String.format("%.2f", h.getAvgBuyPrice()),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 314: `String.format("%.2f", h.getCurrentPrice()),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 315: `String.format("%s%.2f", pl >= 0 ? "+" : "", pl),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 316: `String.format("%s%.1f%", plPercent >= 0 ? "+" : "", plPercent)`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 317: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 318: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 319: `totalValue += h.getCurrentPrice() * h.getQuantity();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 320: `totalPL += pl;`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 321: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 322: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 323: `totalValueLabel.setText(String.format("Total Value: $%,.2f", totalValue));`

Explanation: Updates visible text in a label, button, or field.

Line 324: `totalPLLabel.setText(String.format("Total P/L: %s$%,.2f",`

Explanation: Updates visible text in a label, button, or field.

Line 325: `totalPL >= 0 ? "+" : "", totalPL));`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 326: `totalPLLabel.setForeground(totalPL >= 0 ? PROFIT_GREEN : LOSS_RED);`

Explanation: Sets a Swing component's text color or background color.

Line 327: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 328: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 329: `/** Populate the transactions table. */`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 330: `private void loadTransactions() {`

Explanation: Declares a private helper method used internally by this class.

Line 331: `txnTableModel.setRowCount(0);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 332: `List<Transaction> txns = txnDAO.getRecentTransactions(selectedPortfolio.getPortfolioId());`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 333: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 334: `for (Transaction t : txns) {`

Explanation: Loops over a collection or array to process multiple items.

Line 335: `txnTableModel.addRow(new Object[]{`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 336: `t.getTxnId(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 337: `t.getTicker(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 338: `t.getType(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 339: `t.getQuantity(),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 340: `String.format("$%.2f", t.getPrice()),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 341: `t.getTimestamp().toString()`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 342: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 343: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 344: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 345: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 346: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 347: // ACTIONS

Explanation: Inline comment that explains the intent of the nearby code.

Line 348: // =====

Explanation: Inline comment that explains the intent of the nearby code.

Line 349: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 350: /** Updates simulated prices and refreshes displayed P/L every 10 seconds. */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 351: private void startAutoRefresh() {

Explanation: Declares a private helper method used internally by this class.

Line 352: refreshTimer = new Timer(10000, e -> {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 353: stockDAO.simulateLatestPriceMovement();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 354: refreshDashboard();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 355: });

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 356: refreshTimer.start();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 357: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 358: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 359: @Override

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 360: public void dispose() {

Explanation: Declares a public constructor or method that other classes can call.

Line 361: if (refreshTimer != null) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 362: refreshTimer.stop();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 363: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 364: super.dispose();

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 365: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 366: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 367: `/** Opens the New Trade dialog. */`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 368: `private void openTradeDialog() {`

Explanation: Declares a private helper method used internally by this class.

Line 369: `if (selectedPortfolio == null) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 370: `JOptionPane.showMessageDialog(this,`

Explanation: Shows a Swing dialog message to the user.

Line 371: `"Please select a portfolio first.",`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 372: `"No Portfolio Selected", JOptionPane.WARNING_MESSAGE);`

Explanation: Shows a Swing dialog message to the user.

Line 373: `return;`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 374: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 375: `TransactionDialog dialog = new TransactionDialog(this, selectedPortfolio);`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 376: `dialog.setVisible(true);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 377: `// Refresh data after the dialog closes (trade may have been executed)`

Explanation: Inline comment that explains the intent of the nearby code.

Line 378: `refreshDashboard();`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 379: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 380: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 381: `/** Prompts for a name and creates a new portfolio. */`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 382: `private void createNewPortfolio() {`

Explanation: Declares a private helper method used internally by this class.

Line 383: `String name = JOptionPane.showInputDialog(this,`

Explanation: Shows a Swing dialog message to the user.

Line 384: `"Enter a name for your new portfolio:",`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 385: `"New Portfolio", JOptionPane.PLAIN_MESSAGE);`

Explanation: Shows a Swing dialog message to the user.

Line 386: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 387: `if (name != null && !name.trim().isEmpty()) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 388: `boolean ok = portfolioDAO.createPortfolio(currentUser.getUserId(), name.trim());`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 389: `if (ok) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 390: `JOptionPane.showMessageDialog(this,`

Explanation: Shows a Swing dialog message to the user.

Line 391: `"Portfolio '" + name.trim() + "' created successfully!",`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 392: "Success", JOptionPane.INFORMATION_MESSAGE);
```

Explanation: Shows a Swing dialog message to the user.

```
Line 393: loadPortfolios();
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 394: } else {
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 395: JOptionPane.showMessageDialog(this,
```

Explanation: Shows a Swing dialog message to the user.

```
Line 396: "Failed to create portfolio.",
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 397: "Error", JOptionPane.ERROR_MESSAGE);
```

Explanation: Shows a Swing dialog message to the user.

```
Line 398: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 399: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 400: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 401: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 402: // =====
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 403: // UI HELPERS
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 404: // =====
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 405: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 406: /** Creates a styled flat button. */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 407: private JButton createStyledButton(String text, Color bg, Color hoverBg) {
```

Explanation: Declares a private helper method used internally by this class.

```
Line 408: JButton btn = new JButton(text);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 409: btn.setFont(new Font("Segoe UI", Font.BOLD, 12));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 410: btn.setForeground(Color.WHITE);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 411: btn.setBackground(bg);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 412: btn.setFocusPainted(false);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 413: btn.setBorderPainted(false);
```

Explanation: Sets border/padding styling around a Swing component.

```
Line 414: btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 415: btn.addMouseListener(new java.awt.event.MouseAdapter() {
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 416: public void mouseEntered(java.awt.event.MouseEvent e) { btn.setBackground(hoverBg); }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 417: public void mouseExited(java.awt.event.MouseEvent e) { btn.setBackground(bg); }
```

Explanation: Declares a public constructor or method that other classes can call.

```
Line 418: };
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 419: return btn;
```

Explanation: Returns a value from the current method to its caller.

```
Line 420: }
```

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

```
Line 421: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 422: /** Creates a JTable with dark theme styling and alternating row colors. */
```

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

```
Line 423: private JTable createStyledTable(DefaultTableModel model) {
```

Explanation: Declares a private helper method used internally by this class.

```
Line 424: JTable table = new JTable(model);
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 425: table.setFont(new Font("Segoe UI", Font.PLAIN, 12));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 426: table.setForeground(TEXT_PRIMARY);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 427: table.setBackground(TABLE_ROW);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 428: table.setGridColor(BORDER_COLOR);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 429: table.setRowHeight(30);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 430: table.setSelectionBackground(ACCENT);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 431: table.setSelectionForeground(Color.WHITE);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 432: table.setShowVerticalLines(false);
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 433: table.setInterCellSpacing(new Dimension(0, 1));
```

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

```
Line 434: [blank line]
```

Explanation: Blank line used to separate logical parts of the Java file.

```
Line 435: // Header styling
```

Explanation: Inline comment that explains the intent of the nearby code.

```
Line 436: JTableHeader header = table.getTableHeader();
```

Explanation: Assigns or initializes a value used by the surrounding method or class.

```
Line 437: header.setFont(new Font("Segoe UI", Font.BOLD, 12));
```

Explanation: Sets the font family, size, or weight for a Swing component.

```
Line 438: header.setForeground(TEXT_SECONDARY);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 439: header.setBackground(HEADER_BG);
```

Explanation: Sets a Swing component's text color or background color.

```
Line 440: header.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, BORDER_COLOR));
```

Explanation: Sets border/padding styling around a Swing component.

Line 441: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 442: `// Alternating row renderer with P/L coloring`

Explanation: Inline comment that explains the intent of the nearby code.

Line 443: `table.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 444: `@Override`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 445: `public Component getTableCellRendererComponent(JTable t, Object value,`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 446: `boolean isSelected, boolean hasFocus, int row, int column) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 447: `Component c = super.getTableCellRendererComponent(t, value, isSelected, hasFocus, row, column);`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 448: `if (!isSelected) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 449: `c.setBackground(row % 2 == 0 ? TABLE_ROW : TABLE_ROW_ALT);`

Explanation: Sets a Swing component's text color or background color.

Line 450: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 451: `// Color P/L columns green/red`

Explanation: Inline comment that explains the intent of the nearby code.

Line 452: `String text = value != null ? value.toString() : "";`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 453: `if (text.startsWith("+")) {`

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 454: `c.setForeground(PROFIT_GREEN);`

Explanation: Sets a Swing component's text color or background color.

Line 455: `} else if (text.startsWith("-")) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 456: `c.setForeground(LOSS_RED);`

Explanation: Sets a Swing component's text color or background color.

Line 457: `} else if ("BUY".equals(text)) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 458: `c.setForeground(PROFIT_GREEN);`

Explanation: Sets a Swing component's text color or background color.

Line 459: `} else if ("SELL".equals(text)) {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 460: `c.setForeground(LOSS_RED);`

Explanation: Sets a Swing component's text color or background color.

Line 461: `} else {`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 462: `c.setForeground(TEXT_PRIMARY);`

Explanation: Sets a Swing component's text color or background color.

Line 463: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 464: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 465: `return c;`

Explanation: Returns a value from the current method to its caller.

Line 466: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 467: `});`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 468: `[blank line]`

Explanation: Blank line used to separate logical parts of the Java file.

Line 469: `return table;`

Explanation: Returns a value from the current method to its caller.

Line 470: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 471: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 24: `private static final Color BG = new Color(36, 40, 50);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 25: `private static final Color FIELD_BG = new Color(50, 54, 68);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 26: `private static final Color FIELD_BORDER = new Color(70, 75, 90);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 27: `private static final Color TEXT_PRIMARY = new Color(230, 233, 240);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 28: `private static final Color TEXT_SECONDARY= new Color(150, 155, 170);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 29: `private static final Color BUY_GREEN = new Color(60, 180, 100);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 30: `private static final Color BUY_HOVER = new Color(80, 200, 120);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 31: `private static final Color SELL_RED = new Color(220, 70, 70);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 32: `private static final Color SELL_HOVER = new Color(240, 90, 90);`

Explanation: Defines a reusable UI color constant for consistent styling across this Swing screen.

Line 33: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 34: `private JComboBox<Stock> stockCombo;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 35: `private JTextField quantityField;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 36: `private JTextField priceField;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 37: `private JRadioButton buyRadio;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 38: `private JRadioButton sellRadio;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 39: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 40: `private final Portfolio portfolio;`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 41: `private final StockDAO stockDAO = new StockDAO();`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 42: `private final TransactionDAO txnDAO = new TransactionDAO();`

Explanation: Declares a private field that stores state, a DAO dependency, or a Swing component for later use.

Line 43: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 44: `/**`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 45: `* @param parent The parent DashboardFrame`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 46: `* @param portfolio The portfolio to trade within`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 47: `*/`

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 48: `public TransactionDialog(JFrame parent, Portfolio portfolio) {`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Explanation: Controls the component/window dimensions used by the Swing layout.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Adds a Swing component to a container so it appears in the UI layout.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Assigns or initializes a value used by the surrounding method or class.

Explanation: Blank line used to separate logical parts of the Java file.

Explanation: Inline comment that explains the intent of the nearby code.


```
Line 147: });
```

Line 148: [blank line]

```
Line 149: executeBtn.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
Line 150: public void mouseEntered(java.awt.event.MouseEvent e) {
```

```
Line 151: executeBtn.setBackground(buyRadio.isSelected() ? BUY_HOVER : SELL_HOVER);
```

```
Line 152: }
```

```
Line 153: public void mouseExited(java.awt.event.MouseEvent e) {
```

```
Line 154: executeBtn.setBackground(buyRadio.isSelected() ? BUY_GREEN : SELL_RED);
```

```
Line 155: }
```

```
Line 156: });
```

```
Line 157: executeBtn.addActionListener(e -> executeTrade());
```

Line 158: [blank line]

[illegible]

```
Line 160: card.add(title);
```

```
Line 161: card.add(Box.createVerticalStrut(20));
```

```
Line 162: card.add(stockLabel); card.add(Box.createVerticalStrut(6)); card.add(stockCombo);
```

```
Line 163: card.add(Box.createVerticalStrut(14));
```

```
Line 164: card.add(typeLabel); card.add(Box.createVerticalStrut(6)); card.add(radioPanel);
```

```
Line 165: card.add(Box.createVerticalStrut(14));
```

```
Line 166: card.add(qtyLabel); card.add(Box.createVerticalStrut(6)); card.add(quantityField);
```

```
Line 167: card.add(Box.createVerticalStrut(14));
```

```
Line 168: card.add(priceLabel); card.add(Box.createVerticalStrut(6)); card.add(priceField);
```

```
Line 169: card.add(Box.createVerticalStrut(24));
```

```
Line 170: card.add(executeBtn);
```

Page 107

Line 171: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 172: setContentPane(card);

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 173: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 174: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 175: /**

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 176: * Validates inputs and executes the BUY or SELL transaction via TransactionDAO.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 177: * Displays success/error messages via JOptionPane.

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 178: */

Explanation: Javadoc/comment text. It explains the purpose of the class, method, field, or block for readers.

Line 179: private void executeTrade() {

Explanation: Declares a private helper method used internally by this class.

Line 180: Stock selectedStock = (Stock) stockCombo.getSelectedItemAt();

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 181: if (selectedStock == null) {

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 182: showError("Please select a stock.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 183: return;

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 184: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 185: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 186: // Parse & validate quantity

Explanation: Inline comment that explains the intent of the nearby code.

Line 187: int quantity;

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 188: try {

Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.

Line 189: quantity = Integer.parseInt(quantityField.getText().trim());

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 190: if (quantity <= 0) throw new NumberFormatException();

Explanation: Starts a conditional branch that only runs when the stated condition is true.

Line 191: } catch (NumberFormatException ex) {

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 192: showError("Please enter a valid positive integer for quantity.");

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 193: return;

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 194: }

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 195: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.	
Line 196: <code>// Parse & validate price</code>	
Explanation: Inline comment that explains the intent of the nearby code.	
Line 197: <code>float price;</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 198: <code>try {</code>	
Explanation: Starts a try block for code that can throw an exception, often database or UI setup work.	
Line 199: <code>price = Float.parseFloat(priceField.getText().trim());</code>	
Explanation: Assigns or initializes a value used by the surrounding method or class.	
Line 200: <code>if (price <= 0) throw new NumberFormatException();</code>	
Explanation: Starts a conditional branch that only runs when the stated condition is true.	
Line 201: <code>} catch (NumberFormatException ex) {</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 202: <code>showError("Please enter a valid positive price.");</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 203: <code>return;</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 204: <code>}</code>	
Explanation: Closes or opens a Java block, method, class, lambda, or initializer.	
Line 205: <code>[blank line]</code>	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 206: <code>boolean success;</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 207: <code>String type;</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 208: <code>[blank line]</code>	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 209: <code>if (buyRadio.isSelected()) {</code>	
Explanation: Starts a conditional branch that only runs when the stated condition is true.	
Line 210: <code>type = "BUY";</code>	
Explanation: Assigns or initializes a value used by the surrounding method or class.	
Line 211: <code>success = txnDAO.executeBuy(</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 212: <code>portfolio.getPortfolioId(), selectedStock.getStockId(), quantity, price);</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 213: <code>} else {</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 214: <code>type = "SELL";</code>	
Explanation: Assigns or initializes a value used by the surrounding method or class.	
Line 215: <code>success = txnDAO.executeSell(</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 216: <code>portfolio.getPortfolioId(), selectedStock.getStockId(), quantity, price);</code>	
Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.	
Line 217: <code>}</code>	
Explanation: Closes or opens a Java block, method, class, lambda, or initializer.	
Line 218: <code>[blank line]</code>	
Explanation: Blank line used to separate logical parts of the Java file.	
Line 219: <code>if (success) {</code>	
Explanation: Starts a conditional branch that only runs when the stated condition is true.	

Explanation: Sets the font family, size, or weight for a Swing component.

Line 245: `l.setForeground(TEXT_SECONDARY);`

Explanation: Sets a Swing component's text color or background color.

Line 246: `l.setAlignmentX(Component.LEFT_ALIGNMENT);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 247: `return l;`

Explanation: Returns a value from the current method to its caller.

Line 248: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 249: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 250: `private JTextField createTextField() {`

Explanation: Declares a private helper method used internally by this class.

Line 251: `JTextField field = new JTextField();`

Explanation: Assigns or initializes a value used by the surrounding method or class.

Line 252: `field.setFont(new Font("Segoe UI", Font.PLAIN, 14));`

Explanation: Sets the font family, size, or weight for a Swing component.

Line 253: `field.setForeground(TEXT_PRIMARY);`

Explanation: Sets a Swing component's text color or background color.

Line 254: `field.setBackground(FIELD_BG);`

Explanation: Sets a Swing component's text color or background color.

Line 255: `field.setCaretColor(TEXT_PRIMARY);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 256: `field.setBorder(BorderFactory.createCompoundBorder(`

Explanation: Sets border/padding styling around a Swing component.

Line 257: `BorderFactory.createLineBorder(FIELD_BORDER),`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 258: `new EmptyBorder(8, 12, 8, 12)`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 259: `));`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 260: `field.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));`

Explanation: Controls the component/window dimensions used by the Swing layout.

Line 261: `field.setAlignmentX(Component.LEFT_ALIGNMENT);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 262: `return field;`

Explanation: Returns a value from the current method to its caller.

Line 263: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 264: [blank line]

Explanation: Blank line used to separate logical parts of the Java file.

Line 265: `private void styleRadio(JRadioButton radio) {`

Explanation: Declares a private helper method used internally by this class.

Line 266: `radio.setFont(new Font("Segoe UI", Font.BOLD, 13));`

Explanation: Sets the font family, size, or weight for a Swing component.

Line 267: `radio.setForeground(TEXT_PRIMARY);`

Explanation: Sets a Swing component's text color or background color.

Line 268: `radio.setBackground(BG);`

Explanation: Sets a Swing component's text color or background color.

Line 269: `radio.setFocusPainted(false);`

Explanation: Java statement that contributes to the surrounding class, method, UI layout, DAO query, or model behavior.

Line 270: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.

Line 271: `}`

Explanation: Closes or opens a Java block, method, class, lambda, or initializer.